TEAM SINθ

ENGINEERING JOURNAL

WRO Future Engineers 2026

Aarav Daudia • Harshad Mahajan • Rajas Salunke

Table of Contents

Abstract....4
1.0 Team Profile....5
1.1 Team Photo....5
1.2 The Challenge....5
2.0 Meet The Team....6
Our Engineering Journal....7
4.0 Competitive Goal....7
5.0 Finance....8
6.0 Season Workflow....10
6.1 Team Scheduling & Coach Meetings....10
6.2 Personal & Team Deadlines....10
6.3 Self-Evaluation & Continuous Progress....10
7.0 Game Analysis....11
7.1 Previous Competition Analysis....11
7.2 Strategy & Approach....11
7.3 Our Final Approach....12
8.0 Research & Concept Development....12
8.1 Mechanical Concept....12
8.2 Power & Sensor Architecture....13
8.3 Software Concept....13
8.4 Initial Robot Concepts....13
9.0 Mobility & Mechanical Design....14
9.1 Mechanical Requirements....14
9.2 Chassis....14
9.3 Wheels....16
9.4 Differential....17
9.5 Differential and Axle Holders....18
9.6 Rear Axles....19
9.7 Steering System (Ackermann)....19
9.8 Electronics Plate....23
9.9 Robot Height & Camera Angle Selection....24
Robot Height Reasoning....24
9.10 Motor Adapter....25
9.11 Bumper....25
9.12 Final Mechanical Assembly....25
10.0 Power & Sensor Architecture....27
10.1 Motor Requirements/Selection....27

10.2 Battery Selection..... 28
10.3 Motor Driver Selection..... 29
10.4 PCB Version 3: Final Version..... 30
10.5 Raspberry Pi 4B..... 33
10.6 Sensor..... 33
11.0 Software Architecture and Obstacle Strategy..... 34
11.1 Raspberry Pi + Arduino Nano..... 35
11.2 Vision Pipeline..... 36
11.3 Wall Following..... 38
11.4 Obstacle Avoidance..... 39
11.5 Crash Detection..... 41
11.6 Lap Counter..... 42
11.7 Parking..... 42
11.8 The State Machine..... 43
11.9 Arbitration..... 44
11.10 Software Testing & Iteration Summary..... 45
11.11 Performance & Validation..... 47
12.0 Systems Thinking & Engineering Decisions..... 49
12.1 Trade Offs..... 49
12.2 Constraints..... 50
12.3 Risk & Failures..... 51
12.4 Iterations..... 56
12.5 Safety, Reliability & Known Limitations:..... 58
12.6 Testing & Validation for Reliability..... 59
13.0 Reproducibility & GitHub..... 61
13.1 How to Assemble the Robot..... 61
13.2 How to Build the Custom PCB..... 65
13.3 Software Setup..... 67
Conclusion..... 71
References & Technical Sources..... 71

Abstract

ROBOT

Team SinΘ Robot, a from-scratch autonomous vehicle built for the Open and Obstacle Challenges.

KEY INNOVATION

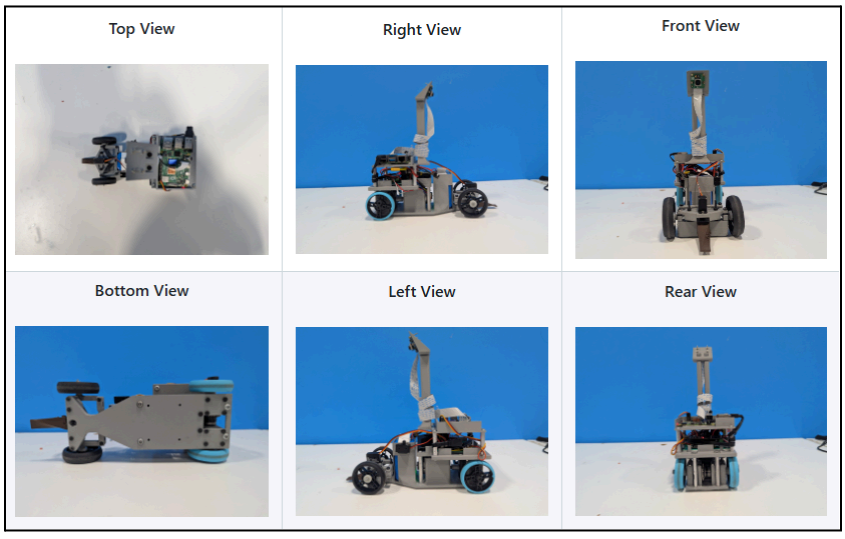
A single-sensor, continuous-motion architecture: one forward-tilted camera (no ToF or gyro) drives every navigation decision through a LAB-colour-space vision pipeline, keeping the sense-think-act loop at roughly 150 ms per frame. Steering decisions are resolved by a priority-based arbitration layer — outer-wall safety, inner-wall damping, obstacle avoidance, then wall-following — running on a Raspberry Pi 4B paired with an Arduino Nano for real-time motor/servo control. The chassis pairs an F1-inspired long-wheelbase layout with calculated Ackermann steering, a metal differential on a direct-drive 510 RPM motor, and a fully modular, independently-serviceable plate stack.

HEADLINE STATS

TOP WHEEL SPEED	SENSORS USED	PROCESSING CYCLE
1.29 m/s	**Pi Camera Module 3 Wide**	**~150 ms / frame**
DRIVING ACCURACY	**LAP TIME**	**ROBOT WEIGHT**
~90% (fixed speed)	**70 s**	**1.005kg**

KEY ACHIEVEMENT

Diagnosed and resolved two full PCB failure cycles, a 7805 linear-regulator thermal shutdown (15.6 W dissipation) and a subsequent motor-driver/servo power-isolation failure, redesigning the electrical architecture around a buck converter and a dedicated servo UBEC to reach a stable, camera-only navigation system built entirely from scratch for $427.76 in parts.



1.0 Team Profile

1.1 Team Photo

Team SinΘ (Aarav, Harshad, Rajas)

About Us:

Hello! We are Team Sin Theta, competing in the WRO Future Engineers 2026 Challenge, bringing together three years of combined robotics experience from previous competition teams. This year, we've unified our expertise to design a high-precision autonomous robot that prioritizes consistency, reliability, and engineering rigor. We previously competed under two teams, The Robertos[aka AA Batteries] (Aarav) and VHR Motors (Harshad, Rajas)

Team VHR Github: https://github.com/RAJAS2025/VHR-WRO-FE-June-2025

Team The Robertos Github: https://github.com/Abdu1Hak/WRO-2025

1.2 The Challenge

The WRO Future Engineers 2026 challenge is an autonomous driving competition where teams design and build a robot capable of navigating a racetrack without human control. The competition consists of two different challenges:

Open Challenge

The robot must autonomously navigate the racetrack and complete three laps, maintaining its position through straight sections and corners. After completing the laps, it must locate the parking area and perform a parallel parking maneuver.

Obstacle Challenge

The robot must navigate the same type of racetrack while also detecting and avoiding randomly placed obstacles. It must use its sensors and vision system to identify the obstacles and choose the correct path around them before completing the required laps and parking.

Goal

The goal of the challenge is to develop a fast, accurate, and reliable autonomous vehicle. It combines mechanical engineering, electronics, computer vision, sensors, programming, and control systems to create a robot capable of making decisions and adapting to its environment in real time.

Scoring & Documentation

In addition to performance on the field, teams are evaluated on their Engineering Journal and GitHub documentation. The 2026 documentation rubric evaluates five areas, each worth up to 6 points, for a total of 30 points:

- Mobility & Mechanical Design — chassis, drive and steering systems, and mechanical decisions
- Power & Sensor Architecture — power system, sensors, wiring, placement, and calibration
- Software Architecture & Obstacle Strategy — code structure, algorithms, navigation, and obstacle handling
- Systems Thinking & Engineering Decisions — testing, design trade-offs, iterations, and engineering reasoning
- Reproducibility & GitHub Quality — clear documentation that allows others to understand and reproduce the robot

The documentation is judged on the quality of engineering reasoning, testing, iteration, and reproducibility, rather than simply its length or visual appearance.

2.0 Meet The Team

Aarav: Hello Judges, my name is Aarav, and I am a grade 12 student with a passion for creating things that blend engineering, technology, and design. Whether I am modeling mechanical parts in CAD, programming robots, wiring electronics, or building websites and machine learning tools, I am drawn to projects that turn curiosity into something real and useful This is my second year competing in WRO FE, with my previous robotics experience coming in WRO FE, VEX, WRO Robosports, FLL, FRC, and Zone_01.

Feel free to check out my new website: **https://crimeinsight.pages.dev/**

Harshad: Hi, I am in Grade 12 and going to John Fraser Secondary School. This is my 2nd year competing in WRO. I'm interested in coding, building robots, and exploring creative ideas. I also enjoy sports and traveling.

Here's my Linkedin: https://www.linkedin.com/in/harshad-mahajan-9580362ba/

Rajas: Hello Im Rajas, I am a student at Fletchers Meadow Secondary School who is currently in Grade 12. This is my 3rd year competing in WRO with experience in Robomission (Junior Category), and 1 year in WRO future engineers with this being my second year in Future Engineers.

Here's my Linkedin: https://www.linkedin.com/in/rajas-salunke-632794422/

Our Engineering Journal

Our Engineering Journal documents the complete development of our WRO Future Engineers robot from our initial ideas and designs to testing, failures, improvements, and our final system. Rather than only presenting what we built, we explain why we made each engineering decision, what we learned from testing, and how our design evolved throughout the season.

The journal is structured around the five areas defined by the WRO Future Engineers 2026 documentation rubric:

- Mobility & Mechanical Design — our chassis, differential, steering system, mechanical design choices, and their development.
- Power & Sensor Architecture — our electrical system, power distribution, sensors, wiring, placement, and calibration.
- Software Architecture & Obstacle Strategy — our software structure, computer vision, navigation, control algorithms, and obstacle-handling strategies.
- Systems Thinking & Engineering Decisions — how our subsystems work together, the constraints we faced, our design trade-offs, testing, failures, and iterations.
- Reproducibility & GitHub Quality — how our code, CAD, wiring, documentation, and project structure allow another team to understand and reproduce our system.

Throughout each section, we focus on the engineering process, not just the final result. We document our reasoning, experiments, test results, problems, and solutions so that our progress can be clearly followed from one iteration to the next.

The Engineering Journal and our GitHub repository work together as a complete record of our project. The WRO guidelines emphasize that the documentation should demonstrate the engineering process, design reasoning, testing and iteration, systems thinking, and reproducibility rather than being judged simply on its length or appearance.

Our goal for this journal is to explain the how and why of our robot and choices.

4.0 Competitive Goal

Our goal for WRO Future Engineers 2026 is to design and build a fully functional autonomous vehicle completely from scratch. While we want to perform competitively and achieve strong results, our main objective is to use this competition as an opportunity to develop our engineering skills and learn through hands-on experience.

Throughout the season, we aim to understand the complete engineering process from mechanical design and electronics to programming, computer vision, control systems, and testing. We want to experience the challenges of building a real autonomous system, learn from our failures, and continuously improve our robot through testing and iteration.

Ultimately, our competitive goal is to create a fast, reliable, and consistent robot that can successfully complete both the Open and Obstacle Challenges while demonstrating the engineering knowledge and skills we have developed throughout the project. This reflects the core purpose of Future Engineers: learning by designing, building, testing, and improving our own solution.

5.0 Finance

5.1 Bill of Materials (BOM)

#	Component	Category	Qty	Line Total
1	Raspberry Pi 4B	Compute	1	$230.95
2	Arduino Nano	Compute	1	$19.95
3	Raspberry Pi Camera Module 3 Wide	Sensing	1	$50.95
4	12V 37D Metal Gearmotor	Actuation	1	$26.88
5	Adafruit DRV8871 Motor Driver	Actuation	1	$7.50
6	8.4V High-Torque Coreless Servo	Actuation	1	$17.00
7	8.4V 10A UBEC	Power	1	$25.99
8	5V DC-DC Buck Converter	Power	1	$2.80
9	IFR 18650 4S1P 12.8V 2000mAh Li-ion Battery Pack	Power	1	$39.99
10	Rear Metal Axles	Actuation	2	$16.50
11	3D Printed Parts Filament	Actuation	1500g	$5.11
Total				**$443.62**

5.2 Miscellaneous Mechanical and Structural Parts (With Cost)

#	Component	Line Total
1	Total Filament Cost Across all Iterations (PLA, PETG)	$15.00
2	VEX Standoffs (1”, 1 ½”)	$1.50
3	Bearings (*4)	$1.00
4	Screws, Fasteners (M2, M3)	$1.50
5	VEX Nylon Spacers (¼”, ⅛”)	$0.25
6	SPIKE Wheels (*4)	$8.00
7	Metal Differential	$8.00
	TOTAL	**$35.25**

5.3 Budget vs. Actual

Category	Budgeted	Actual	Variance
Compute	$300	$250.9	$49.1
Sensing	$50	$50.95	-$0.95
Actuation	$75	$72.99	$2.01
Power	$75	$68.78	$6.22
TOTAL	**$500**	**$443.62**	**$56.38**

6.0 Season Workflow

6.1 Team Scheduling & Coach Meetings

We created a structured schedule to make sure progress continued throughout the season. At least **two team** members worked on the **robot** every day, allowing us to divide responsibilities and keep development moving even when the full team was unavailable.

We held consistent meetings with our coaches to review our progress, discuss challenges, and receive feedback. These meetings helped us stay on track and make better engineering decisions throughout the season.

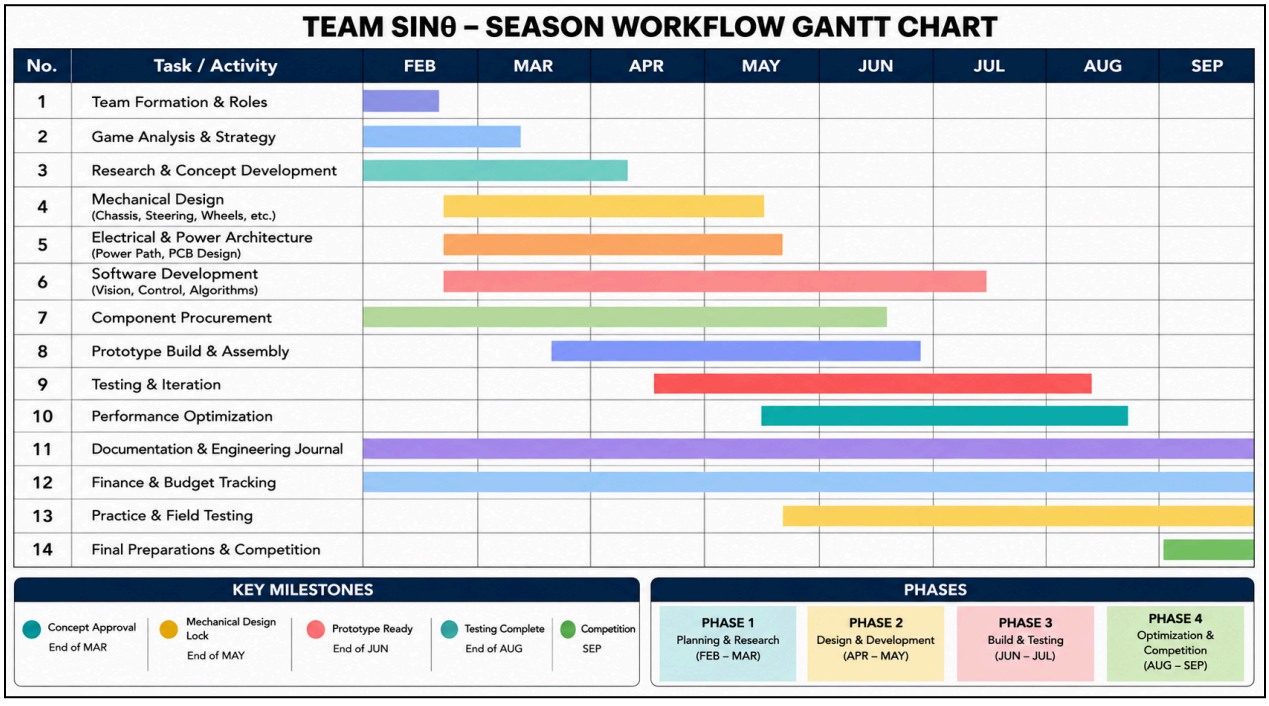
6.2 Personal & Team Deadlines

To prevent tasks from being left until the last minute, we set personal and team deadlines for major milestones. This helped us manage our time, divide the workload, and ensure that each part of the robot was completed on schedule.

6.3 Self-Evaluation & Continuous Progress

Throughout the season, we regularly reviewed and evaluated our progress. We compared our robot and documentation against the competition requirements and identified areas that needed improvement. This allowed us to make changes early and continuously improve our work.

Our workflow followed a **cycle of planning → building → testing → evaluating → improving**. By maintaining consistent progress and holding ourselves accountable, we were able to manage our time effectively while giving ourselves enough time to properly test and refine the final robot.



7.0 Game Analysis

7.1 Previous Competition Analysis

We analyzed the performance of both of our teams from the previous WRO Future Engineers competition to identify what **worked well**, what could be **improved**, and what lessons we could carry into the 2026 season.

	Team 1 (VHR Motors — Harshad & Rajas)	Team 2 (The Robertos — Aarav)
Strengths	Steering System (sharp steering) Gearing (able to go fast even with low RPM motor) Organization of Code (comments, easy to understand)	Modular chassis (easy to assemble and repair) Use of Raspberry Pi 4B (able to use OpenCV for more effective image processing) Metal Differential (accurate, strong)
Weaknesses	Huskylens Camera (bad for colour detection, no flexibility to customize image processing) Bad centre of gravity (robot tipping) Wheel skidding during turns (no Ackermann, used parallel steering) ToF Sensors (slowed code and decision making) Monocoque Chassis (difficult for repairs) 3D Printed Differential (good, but weak and not accurate)	Low motor torque and RPM Multiple fragile components
Lessons Learned	We learned that sharp steering, gearing, and well-organized code can greatly improve performance. However, issues with the centre of gravity, wheel grip, motor performance, and the monocoque chassis showed us the importance of stability, serviceability, and drivetrain design.	We learned that a modular chassis, Raspberry Pi with OpenCV, and metal differential provided a strong foundation for the robot. However, the complex code, slow drivetrain, and fragile 3D-printed components showed us the importance of efficient software, speed, and stronger mechanical components.

7.2 Strategy & Approach

Our Goal

Our main strategy is to build a robot that does not need to stop during the challenge. We want it to continuously navigate the track while making decisions in real time, allowing us to maintain high speed without sacrificing accuracy or consistency.

Our approach focuses on three main goals:

- **Accuracy** — reliably follow the track and make correct decisions.
- **Consistency** — perform reliably across repeated runs and different track configurations.
- **Speed** — maintain a high speed while remaining stable and controlled.

Inspiration From Other Teams

As part of our research, we studied other WRO Future Engineers teams to understand different approaches to strategy, software, mechanical design, and documentation. These teams influenced our own decisions and helped us identify practices that we could adapt to our robot.

Team	What Inspired Us
Double XX	We liked their simple and effective strategy, as well as how their software was kept simple, easy to understand, and accurate. Their GitHub was also straightforward and focused on reproducibility, making it easy to understand how their robot worked.
Storms NGR	We were inspired by the level of detail and accuracy in their GitHub documentation. Their explanations made their engineering decisions and implementation easy to follow.
Greenbotics	We liked their use of servo clamping, which improved the efficiency and consistency of their robot. This inspired us to focus on solutions that make our own robot smoother, more efficient, and more consistent.
KMIDS GFM	We liked the depth of their GitHub documentation, particularly their use of photos and GIFs to demonstrate their robot and engineering process. This influenced how we present our own work visually.

7.3 Our Final Approach

Combining what we learned from our previous robots, other teams, and our own experience, we developed an approach focused on continuous movement. Our goal is not simply to make the fastest robot, but to create a system that can drive quickly, make accurate decisions, and repeat its performance consistently.

8.0 Research & Concept Development

Before beginning construction, we carried out initial research to understand the challenge, identify the requirements of our robot, and explore possible solutions. We used this research to develop our concepts and make informed engineering decisions before moving into detailed design.

8.1 Mechanical Concept

We researched different chassis layouts, steering systems, wheel configurations, drivetrains, and differentials to determine what would best suit our robot. We were also **inspired by Formula 1 cars**, particularly their relatively **large length-to-width ratio**. A longer chassis can provide a larger turning radius and more predictable movement through corners, which influenced the proportions of our design.

We then developed our initial mechanical concept based on these findings, focusing on stability, steering accuracy, speed, and serviceability.

8.2 Power & Sensor Architecture

We researched the electrical requirements of the robot, **including battery selection, motors, motor drivers, voltage regulation, sensors, and power distribution**. We also compared different distance and vision sensors to determine which combination would provide reliable information about the robot's surroundings while remaining efficient and practical.

Our goal was to create a power and sensing system that could reliably support every subsystem without unnecessary complexity.

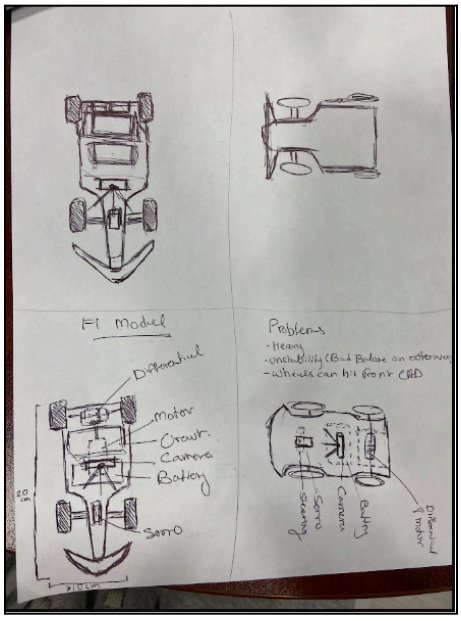
8.3 Software Concept

We researched different approaches to **computer vision, sensor processing, navigation, and control systems.** This included exploring how camera data could be processed to detect track elements and how sensor information could be combined to make accurate driving decisions.

Our software concept was **designed around real-time decision making, with the goal of keeping the robot moving continuously while maintaining accuracy and consistency**.

8.4 Initial Robot Concepts

Before creating our final CAD model, we created **initial sketches and concepts to visualize the overall layout of the robot.** These sketches helped us determine the placement of major components and identify potential design problems before manufacturing.



Initial concept sketches

9.0 Mobility & Mechanical Design

9.1 Mechanical Requirements

Competition constraints (WRO Future Engineers)	Our own design targets
• **Max size**: 300mm x 200mm • **Max height**: 300mm • **Max weight:** 1.5kg	• Compact footprint • Balanced centre of gravity • Tight component tolerances • Easy to assemble, repair, and modify
Design priority order**:** Compactness → Stability → Precision → Serviceability	

9.2 Chassis

9.2.1 Chassis Dimensions — F1-Inspired Layout

Component selection came first: the motor, differential, camera, and control board were chosen based on their real dimensions, and the chassis was then built around them. The F1-inspired ratio (below) was the target shape we designed toward, not a fixed starting constraint.

9.2.2 Component Placement (locked before final chassis size)

- **Motor mount** — low, central, in-line with the differential (no idler gears needed), akin to an M/R layout F1 car, ideal for stability and turning.
- **Battery holder** — centred in middle of chassis, keeping pack mass near the centreline

Final size: 263mm (L) x 105mm (W) — well inside the 300mm x 200mm limit. This reflects real clearance around those components, with margin left for later corner trims.

We based the length-to-width ratio on F1 chassis philosophy: long wheelbase, narrow track, for stability without a large footprint.

Dimension	Value	Effect
Width	105mm	Centres mass on the longitudinal axis, reduces roll/weight-transfer during cornering, adds wall clearance

Dimension	Value	Effect
Length (wheelbase)	263mm	Improves straight-line stability; smaller steering angle needed for the same turning radius

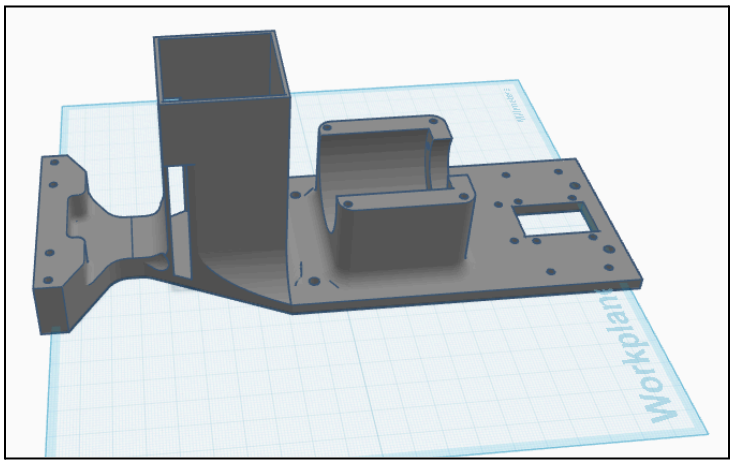


Figure 1 — Chassis CAD

9.2.3 Chassis Architecture

We compared a monocoque chassis (1 piece) with a modular chassis (multi-piece).

Option	Advantages	Disadvantages
Monocoque	Rigid, fewer fasteners, slightly lighter	Repairing one part often requires disassembling or reprinting the whole structure
Modular	Individual sections can be swapped/redesigned independently; faster iteration	More joints/fasteners, small weight penalty

One of our prior teams (VHR Motors) ran a monocoque chassis. A single drivetrain fault required partially disassembling unrelated sections to reach it, costing repair time during test cycles.

Furthermore, team The Robertos robot featured a modular design from which it benefitted during its time (facilitating easy updates and swapped components), thus championing this year's design.

9.2.4 Additional Features

1) The nose is raised to level the front and back axles with a cutaway, providing room for the servo output to steer.
2) The battery holder is **mounted centrally and low to prevent tipping as well as to reduce the weight in the front to improve steering and stability, maintaining overall balance**. It also features cutaways in the front corners to reduce tire rub and wear.

3) The motor is mounted centrally, **the core and the heaviest bit of the robot, everything was built around it**. Having it mounted centrally with a rear-wheel drive likens it to an M/R layout race car, which is designed for stability and balance.
    a) While the team Robertos last year utilized a vertically mounted motor to reduce the space it takes up, this year it was unfeasible due to its sheer size.
    b) The motor holder itself is **circled with six mounts for the motor**, as well as being split in two to allow the motor to be easily placed, held down with four securing screws.
4) The rear is **dotted with mounting points for the rear-axles and differential supports,** with a large cut away rectangle in the middle-rear to allow the differential room to spin freely.

This open-architecture approach allows us to iterate the drivetrain, steering, and electronics plates independently. This mattered more than the small weight saving, since our season depends on running frequent design–test iterations.

9.3 Wheels

Both prior teams (VHR Motors and The Robertos) ran small front wheels and large rear wheels.

Figure 2a — Reference robot: small front wheels, large rear wheel

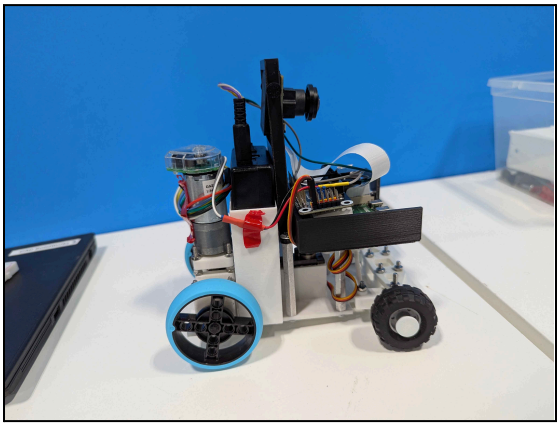


Figure 2b — The Robertos: same mismatched setup

Setup	Rear	Front
The Robertos	57mm	35mm
VHR	40mm	15mm
Sin Theta	57mm	57mm

For The Robertos, the concept of mismatched wheel was in order to produce positive rake to benefit the camera's view, but this year we simply furthered the tilt on the camera to counter this design philosophy.

Both teams reported oversteer under hard cornering with the mismatched setup, caused by the size difference shifting rolling radius and centre-of-gravity height front-to-back.

We selected the LEGO SPIKE Prime wheel (57mm x 14mm) on all four corners. Same size front and rear keeps CG height level and lets one turning-radius table apply to both axles in our Ackermann calculations.

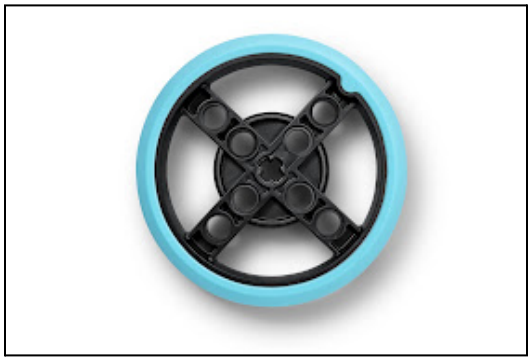


Figure 2c — LEGO SPIKE Prime wheel (57mm x 14mm) used on all four corners

Trade-off: VHR's larger rear wheel gave a higher top speed per motor rotation. We gave that up for consistent handling front-to-back.

9.4 Differential

We compared three options for transferring power between the rear wheels:

Option	Advantages	Disadvantages
3D-Printed	Easy to customize and manufacture	Lower strength and accuracy
LEGO	Easy to assemble and modify	Limited strength for our drivetrain
Metal	Strong, precise, smooth, and durable	Heavier and less customizable

Our motor connects directly to the differential — no gearing. VHR's prior robot used a 1.5:1 gear stage (36-tooth spur into a 24-tooth input gear) on a 175 RPM motor specifically to step RPM up before the differential; while team The Robertos ran a direct-drive design from a vertically mounted motor to reduce size, thus simplifying the design, a concept that carried over as this year we are running a 510 RPM motor directly driven into the differential.

Speed/torque comparison (no-load, **56mm wheels**):

Setup	Motor	Output Speed	Wheel Surface Speed	Torque at Differential
The Robertos - Direct	236 RPM	236 RPM	π × 0.056m × 236 ≈ 42m/min (0.7 m/s)	Full Motor Torque (5.2 kg*cm)
VHR — Geared	175 RPM + 1.5:1 gearing	175 × 1.5 ≈ 263 RPM	π × 0.056m × 263 ≈ 46 m/min (0.77 m/s)	Reduced by gear ratio
Our robot — Direct Drive	510 RPM	510 RPM	π × 0.056m × 510 ≈ 90 m/min (1.5 m/s)	Full motor torque, no reduction (4 kg*cm)

Torque tradeoff: **the 510 RPM motor gets us a higher wheel speed than VHR's geared setup without adding a gear stage**, but a motor spinning this much faster typically trades away stall torque compared to a slower motor of similar size — so we're **relying on the differential to deliver that torque to the wheels with zero mechanical loss instead of gaining it back through reduction gearing.**

Test result (VHR, 1.5:1 gear stage into a 3D-printed differential): measurable gear/housing wear across the season, with mounting-hole play increasing under cornering reaction loads. The Robertos' metal differential ran a full season with no comparable wear.

We selected a metal differential on direct drive with the 510 RPM motor — removing the gear stage removes VHR's identified wear point entirely, and the metal housing adds durability on top of that, as well as weight and unnecessary space, while still beating VHR's geared wheel speed without the extra mechanical complexity.

9.5 Differential and Axle Holders

- The differential maintains its own bearings that the holders are designed to carry it from, thus holding it at the appropriate placement in reference in both the shaft height and the difference in space from the input gear.
- The axle holders are similar in concept, except bearings are fitted within them, snuggly supporting the axles to reduce friction.
- In addition, a single holder for the differential input gear is used to hold it in place. This was fitted as earlier revisions suffered from vibrations and slipping due to the load from the motor.

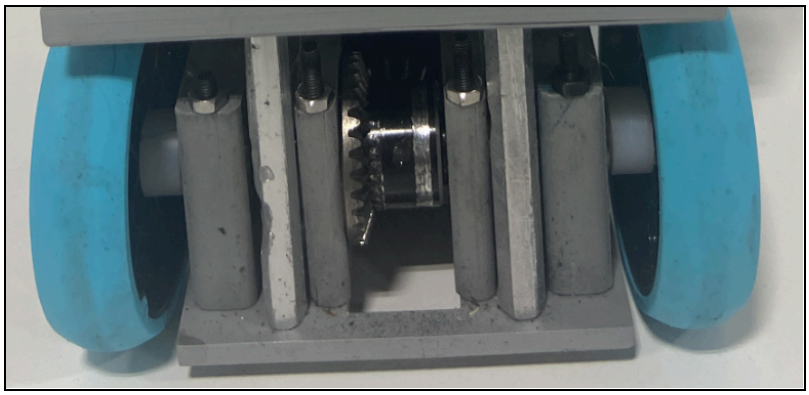


Figure 3 — Metal differential mounted between the rear wheels

9.6 Rear Axles

We tested 3D-printed vs metal rear axles under motor torque:

Axle Material	Flex Under Load	Alignment Shift
3D-Printed	0.8 mm	~2.5° during acceleration; snapped after repeated runs
Metal	0.2 mm	Within tolerance; Minimal twisting

Last year, team Robertos ran a similar design to these current axles, however they were plagued with weakness and fragility due to their 3D printed nature. Thus, **we collaborated with a metal 3D printing service to get these parts custom printed in stainless steel (specifically BJ-316L), greatly increasing their strength.**

The axles have VEX-issue ¼" nylon spacers between them and the wheels to prevent the wheels from moving untowardly during the run, as well as with **LEGO rubber fittings attached with a LEGO black peg to secure the wheel.**

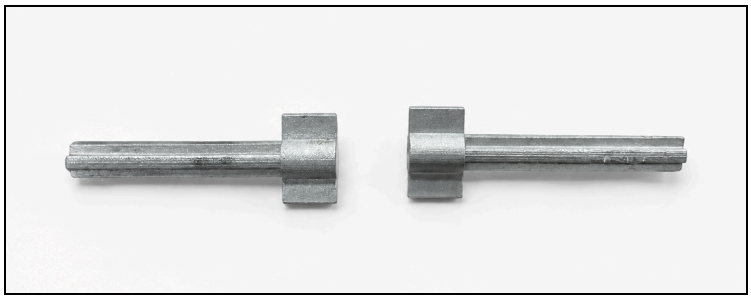


Figure 4 — Metal axles

9.7 Steering System (Ackermann)

In the previous years, **team Robertos utilized a double-arm steering system** which allowed the servo to be mounted in a higher-rear position, it allowed for superior maximal turning angle, but was extremely fragile as well as taking up a large amount of real estate.

Team VHR ran parallel steering last year (both front wheels turn to the same angle). Reported result: inner wheel scrubs against its natural arc on sharp corners, causing skidding and traction loss where the angle mismatch is largest.

We selected front-wheel Ackermann steering, calculated from our chassis dimensions:

- Inner wheel angle: 12°
- Steering pivots positioned so both wheel axes intersect at the correct turning centre

This removes the scrubbing losses reported with parallel steering, at the cost of a more complex linkage geometry to manufacture and align.

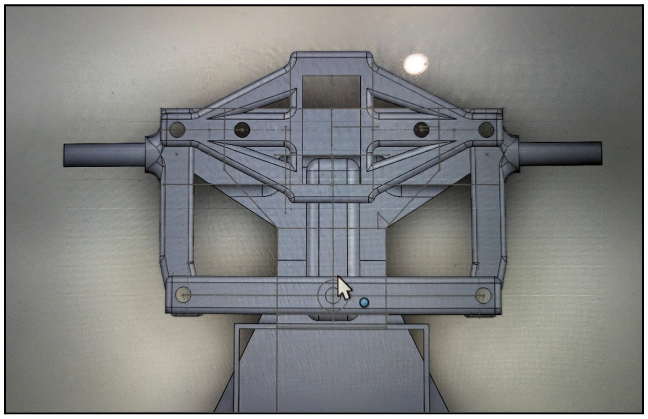
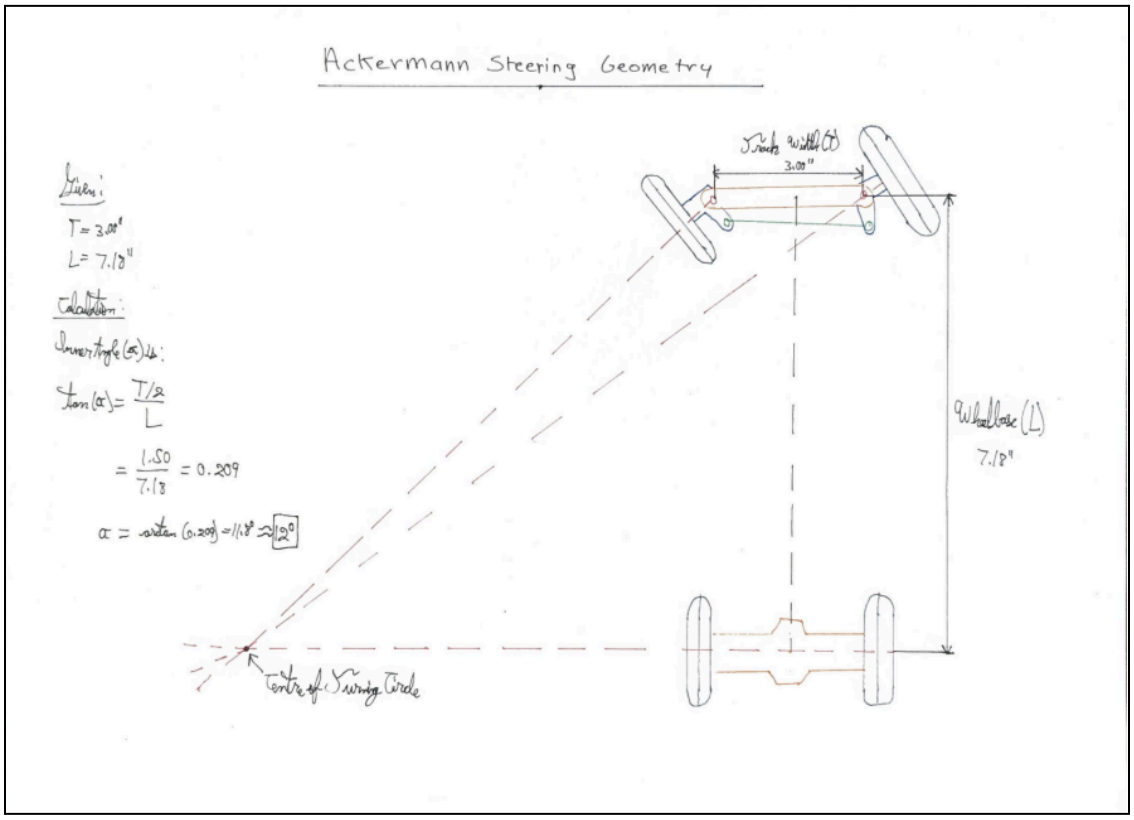
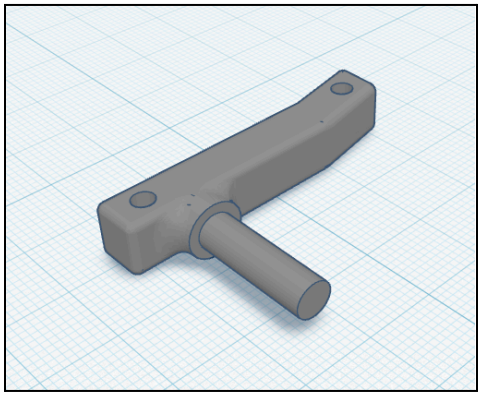


Figure 5 — Ackermann steering linkage CAD, front view

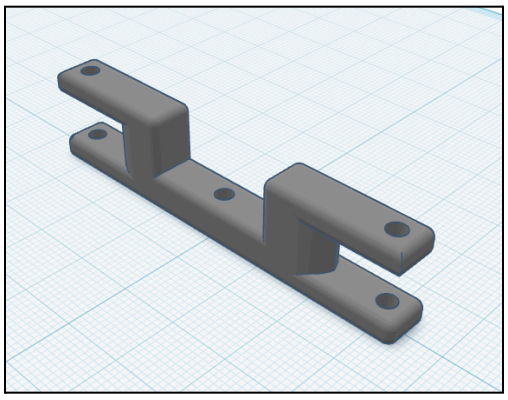


Ackermann Steering Calculations

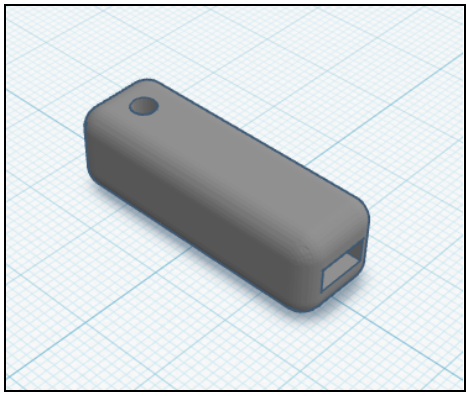
The front wheels are attached to a steering knuckle, which **features a 12 degree angle to produce the 100% ackermann.** For reference, a 100% ackermann design simply means that the lines drawn from each wheel meet directly at the centre of the real axle. A 95% ackermann would mean they meet 5% of the length-wise distance between axles further forward from the rear axle.



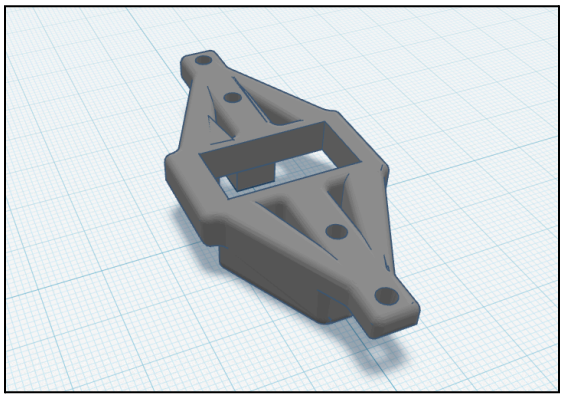
The steering knuckles are attached together by a steering cross link, which attaches to the knuckles from above and below to ensure they are properly fitted together.



The servo output is slipped into a steering T which allows its length to be extended (to meet the longer distance to the steering cross link, which was appended to maximize steering angle), while simultaneously providing a more flush attachment point with minimal friction.



The entire steering assembly is held together by a case that maintains the steering knuckles beneath it. It allows the servo to be mounted inversely to allow the servo output and steering knuckle to be on the same plane, with output protruding underneath, thus reducing complexity and increasing compactness. It featured two mounting points to hold the knuckles to it and the knuckles to the chassis, as well as two mounting points to secure it to the chassis. The part itself features a six-triangle design to both increase strength as well as to reduce weight.



9.8 Electronics Plate

Second plate: holds the custom motor-control PCB. USB serial connection to the Raspberry Pi stays accessible in every mounting orientation, with clearance to swap components without removing the plate above.

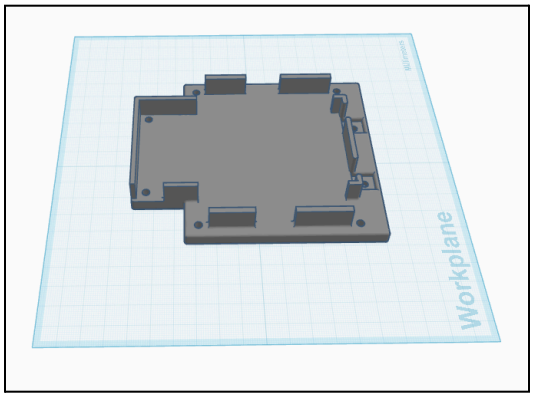


Figure 6 — Second plate CAD, motor-control PCB mount

Third plate: Raspberry Pi Holder + camera mount combined into one rigid piece (not separate brackets), so vibration/collision doesn't shift the pixel-to-real-world mapping the vision system depends on.

The main camera mount is an independent part that allows quick interactions to be made as selecting a camera angle is a tiresome process and thus having to reprint the camera mount along with the Raspberry Pi holder would simply waste time and filament. The camera mount features a 155 degree tilt with flat mounts in the rear to provide flush mounting points for the securing m3 nuts and bolts.

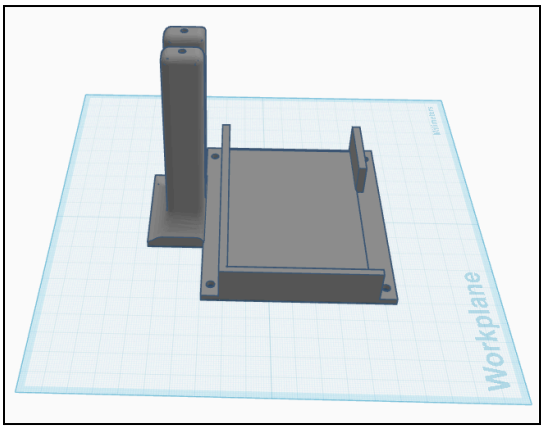


Figure 7 — Third plate CAD, combined Raspberry Pi and camera mount

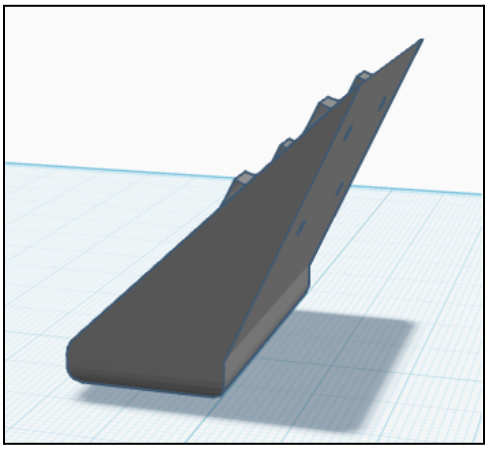


Figure 8 — Camera Mount

9.9 Robot Height & Camera Angle Selection

Robot Height Reasoning

Parameter	Value	Reasoning
Total robot height	268mm	50mm under the 300mm limit, maximizes camera look-ahead without raising CG enough to affect cornering
Camera tilt	155°	Wide enough to keep both walls in frame for PD wall-following; shallow enough to keep the immediate line visible at the bottom of frame

Camera Angle Testing and Reasoning

Degree of TIlt (Degrees)	Near Obstacle Visibility	Far Obstacle Visibility	Overall Field Visibility	Chosen (Yes/No)
135	Excellent	None	Poor, can only see near	No
145	Good	Limited	Okay, sees further obstacles very late	No
155	Good	Very Good	Superior	Yes
165	Limited	Very Good	Good, however stops seeing obstacles too soon, therefore crashes	No
175	Poor	Excellent	Poor, cannot see in front of itself, always crashing	No

Camera Placement — Front vs Middle vs Rear

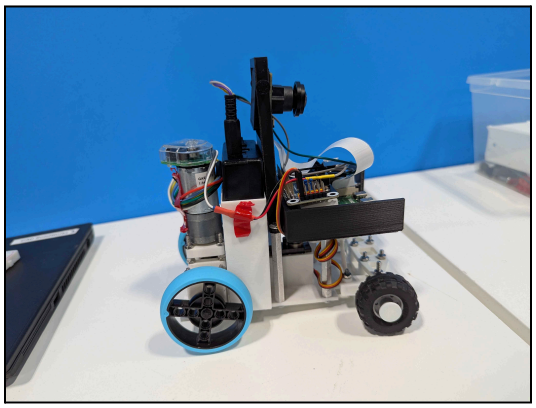


Figure 9 — The Robertos' robot, camera mounted centrally over the chassis

Prior team The Robertos mounted their camera centrally over the chassis rather than at the front or back. We adopted the same placement: **Middle placement keeps the camera equidistant from both side walls through a turn.** VHR's rear mount added lag between what the camera sees and where the front wheels actually are. A front mount avoids that lag but sits too close to the walls in tight corners. We went with the Robertos' middle placement since it avoids both problems.

9.10 Motor Adapter

Simple motor adapter to connect the motor to the metal differential. **The motor maintains a D-shaft output while the differential input gear requires a double D-shaft, therefore the axle adapter is necessary.** It has slots to firmly secure the input gear in place as well as to maximize the rotational motion transferred. **It was printed in PETG to be more tough and prevent grinding.**

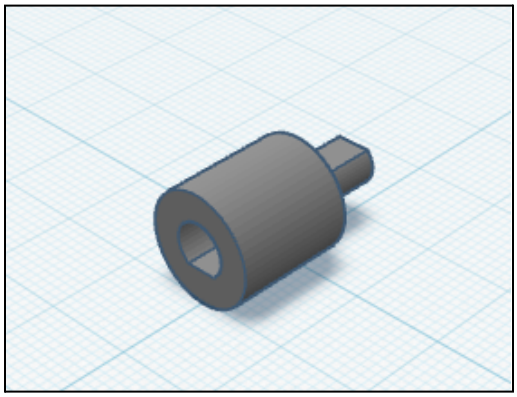


Figure 10 — Motor Adapter

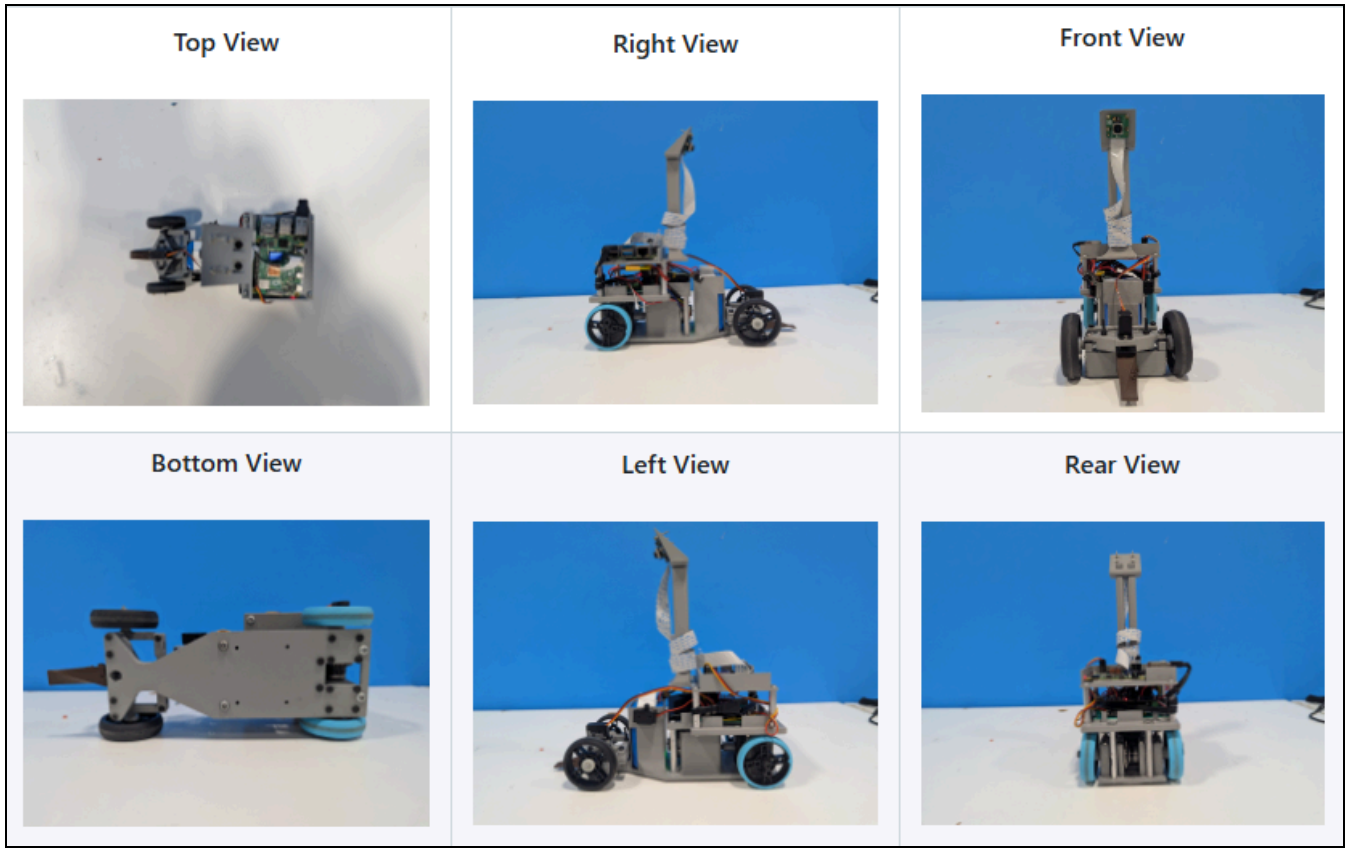
9.11 Bumper

Our robot features a protruding point from the nose of the robot, serving a multitude of purposes:

1) Prevents the robot from becoming stuck on any wall or obstacle.
2) The thick tape securing the bumper acts as a shock absorber to reduce stress on the chassis and rest of the robot during crashes.
3) And of course, a F1 inspired car must have a front bumper so we had to add it, sporting our team name “Sin Theta”, letting everyone know which team is running for.

9.12 Final Mechanical Assembly

Final platform: modular chassis + uniform SPIKE Prime wheels + metal differential + metal rear axles + calculated Ackermann steering + integrated electronics/camera plate stack.



Top / Right / Front / Bottom / Left / Rear views of the final assembly

10.0 Power & Sensor Architecture

This is our Power and Sensor Architecture, where we describe all our components of our **current electrical system** as well as **how we chose them.**

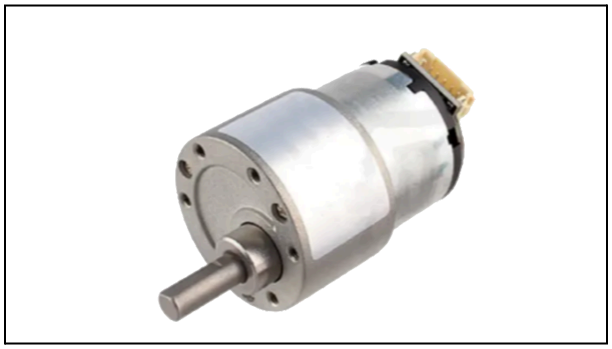
10.1 Motor Requirements/Selection

Using the performance **criteria** we set for our **drive motor, encoder feedback, RPM, torque, gearbox, physical size, and wheel compatibility,** we compared two candidate motors and selected the 12V 37D Metal Gear Motor for meeting every requirement, while the 520 DC Gear Motor fell short on size and output characteristics.

Motor	Encoder (Required)	Rated RPM (250 to 500 target)	Torque (1.5 to 2.0 kg·cm)	Gearbox (Required)	Physical Size (Fits reduced chassis)	Wheel Compatibility (56mm x 14mm)	Chosen?
12V 37D Metal Gear Motor	Yes, integrated encoder for closed loop feedback	Rated: 285 RPM (within target range). No load: 510 RPM, giving a theoretical top wheel speed of approximately 1.5 m/s once unloaded	Above minimum, sufficient torque without running near stall	Yes, built in gearbox	Yes, compact 37D form factor fits within our reduced chassis, leaving room for battery, PCB, sensors, and steering	Yes, directly compatible	**Yes.** Meets every criterion: integrated encoder, rated RPM within our 250 to 500 target, adequate torque, built in gearbox, compact size, and direct wheel compatibility.
520 DC Gear Motor w/ High Precision Hall Encoder	Yes, includes an encoder	Weaker fit. Output characteristics don't match our target range as closely	Weaker fit. Less suited to our torque range	Yes, geared	Weaker fit. Size made it a weaker match for our compact chassis constraints	Not directly compared	**No.** Also has an encoder, but its size and output characteristics made it a weaker overall fit than the 37D motor for our compact chassis and target RPM/torque range.

Based on the physical and electrical demands of the robot, we set performance criteria for our drive motor and compared two candidates against them:

12 V 37D Metal Gear Motor



10.2 Battery Selection

Our **12V 37D Motor** and **DRV8871** driver are both rated for a **12V nominal supply,** so we targeted ≈ 12V as our battery voltage, enough to hit the motor's rated 293 RPM without a boost converter, while staying within the DRV8871's safe input range. We chose a **4S1P Li-ion pack** (12.8V nominal, 2000mAh), since 4 cells in series was the minimum to clear 12V with headroom for voltage sag. With the system drawing up to 2.0A at peak, this capacity and discharge rating held the rail stable through full-load acceleration and steering, with no brownouts on the logic or motor side.

We compared four candidate batteries against this requirement:

Battery	Required Voltage	Supplied Voltage	Size	Chosen?
7.2 V NiMH Battery Pack (6 cell)	12.0 V	7.2 V	Compact, but low energy density for its weight	**No.** Falls far short of our 12 V target. Running our 12 V rated motor at 7.2 V would significantly cut its speed and torque below what our RPM/torque requirements called for, and NiMH chemistry is also heavier per Wh than Li-ion.
9.6 V NiMH Battery Pack (8 cell)	12.0 V	9.6 V	Larger and heavier than the 6 cell pack for a modest voltage gain	**No**. Still 2.4 V short of our 12 V target, and adding the extra NiMH cells needed to close that gap would have made the pack noticeably bulkier and heavier than an equivalent Li-ion option, without matching Li-ion's energy density.
12.8 V 2000 mAh Li-ion Battery Pack (4S1P)	12.0 V	12.8 V	Compact 4 cell in series pack; fits within our reduced chassis alongside the PCB, sensors, and steering system	**Yes**. Its 12.8 V nominal output is the closest match to our 12 V requirement of any candidate, directly driving our motor and DRV8871 without a boost converter. Li-ion's higher energy density than NiMH also kept the pack lightweight and compact enough to fit our smaller than max chassis.

Battery	Required Voltage	Supplied Voltage	Size	Chosen?
18.5 V 5S LiPo Battery Pack (2200 mAh)	12.0 V	18.5 V	Bulkier and heavier than the 4S1P pack due to the extra series cell	**No**. At 18.5 V, this pack is more than 6 V above our ≈12 V target which is far outside the range we needed. Supplying voltage this far above our requirement risks damaging our system by overdriving components that were sized around a 12 V supply, since running everything at 50%+ over our target voltage pushes current and heat well past what our system was designed to handle.

This battery choice carried through unchanged to our final electrical architecture (see below): every later PCB revision was designed around this same 12.8 V supply.

12.8 V 2000 mAh Li-ion Battery Pack (4S1P)

10.3 Motor Driver Selection

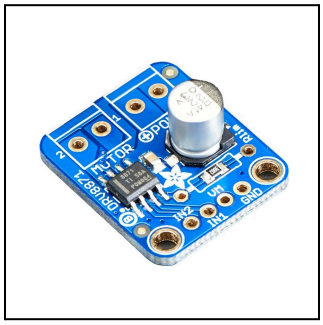
Here is our motor driver selection process:

Motor Driver	Properties	Chosen?
Adafruit DRV8871	6.5–45 V supply, 3.6 A continuous / 6 A peak, PWM speed + direction control, built in overcurrent/thermal/undervoltage/short circuit protection, compatible with 3.3 V logic	**Yes.** Supports our 12.8 V battery (well within its 6.5–45 V range). Its 3.6 A continuous rating exceeds our battery's 2 A max output, so the driver isn't the limiting factor. Compatible with 3.3 V logic (no level converter needed), and its MOSFET design minimizes voltage drop, letting the motor receive nearly the full battery voltage.
Pololu VNH5019 Motor Driver Carrier	5.5–24 V supply, 12 A continuous / 30 A peak, current sensing, reverse voltage + thermal protection	**No**. Compatible voltage wise, but its 12 A rating is six times our battery's 2 A max output, excess capability we can't use, at extra cost.

Motor Driver	Properties	Chosen?
L298N Dual H Bridge Motor Driver	5–35 V supply, 2 A/channel, bipolar transistor H bridge	**No**. Operable from 12.8 V, but has an internal voltage drop of ≈ 2 V, so the motor would only receive ≈ 10.8 V, reducing speed and torque. Also less efficient than MOSFET based drivers.
Cytron MD13S Motor Driver	6–30 V supply, 13 A continuous, MOSFET H bridge, built in protection	**No**. Fully compatible, but its 13 A rating is over six times what our 2 A battery can supply, unnecessary capability and cost.

We selected the Adafruit DRV8871 for its efficient MOSFET switching, effective heat dispersion, and built in protection features.

Adafruit DRV8871 Motor Driver



10.4 PCB Version 3: Final Version

Our shared logic electronics (Nano, sensors, remaining logic) draw up to 1.5A peak at 5V (4.0W). The 7805 linear regulator hit **thermal shutdown** at this load converting 12.8V→5V, so we replaced it with a **5V buck converter** rated for the same 1.5A peak, which dissipates far less heat at that current. The **servo** separately draws up to 3.5A peak (2.8W output) under load, so we isolated it on a dedicated **8A UBEC** rather than sharing the buck rail. After the swap, the board stayed within its thermal limits under sustained current and the servo steered under load without stalling.

10.4.1 PCB Version 3 Components

Component	Description
Arduino Nano	**Slave controller.** Receives steering and speed values from the Raspberry Pi over serial and converts them into PWM signals for the motor driver and servo. Runs on the 5 V rail from the buck converter.
Raspberry Pi 4B + Pi Camera Module 3 Wide	**Main processing unit.** Runs computer vision on the camera feed to make navigation decisions, then sends steering and speed values to the Nano. Powered from the 5 V buck converter rail. See 10.4 for full detail.

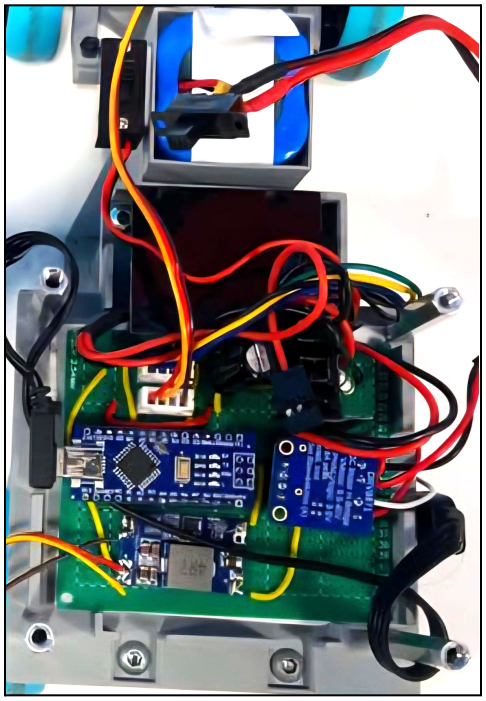
Component	Description
Adafruit DRV8871 Motor Driver	**Drives the 12 V 37D metal gear motor**. Powered directly from the 12.8 V battery rail, with its 3.3/5 V logic side fed from the Nano's PWM outputs.
5 V DC to DC Buck Converter	**Steps the 12.8 V battery voltage down to 5 V** for the Nano, motor encoder, and any other shared logic electronics. Replaces both 7805 regulators used in earlier PCB versions.
2 to 8S LiPo UBEC 8A	**Dedicated 8.4 V regulator that powers only the steering servo,** isolated from the rest of the electronics so servo current spikes can no longer brown out the board.
12.8 V 2000 mAh Li-ion Battery Pack (4S1P)	**Main power source** for the whole board. Feeds the DRV8871, buck converter, and UBEC directly at 12.8 V.
9g Coreless Servo (Full Metal Gear, 4 KG·cm, 4.8–8.4 V)	**Steering actuator.** Powered by the UBEC at 8.4 V, controlled by a PWM signal from the Nano.
12 V 37D Metal Gear Motor	**Drive motor.** Powered directly from the battery through the DRV8871, controlled by PWM signals from the Nano.
Motor Encoder Port	**Connects the drive motor's encoder** back to the Nano for closed loop speed and distance feedback. Powered from the 5 V buck converter rail.
Servo Port	**Dedicated 3 wire connector** (power, ground, PWM) linking the UBEC and Nano to the steering servo.
M1 and M2 Motor Output Ports	Connect the DRV8871's outputs to the drive motor. Carry 12.8 V motor current directly from the battery rail, not through either regulator.

10.4.2 PCB Version 3 Power Budget

Component	Input Voltage	Normal Current	Peak Current	Power
12.8 V 2000 mAh 4S1P Li-ion Battery	12.8 V	1.2 A	2.0 A	15.36 W
DRV8871 Motor Driver	12.8 V	< 0.05 A	< 0.1 A	0.64 W
12 V 37D Metal Gear Motor	12.8 V	0.40 A	1.00 A	5.12 W
5 V DC to DC Buck Converter	12.8 V → 5 V	0.8 A (out)	1.5 A (out)	4.0 W (out)
Arduino Nano	5 V	0.05 A	0.10 A	0.25 W
Raspberry Pi 4B + Pi Camera 3 Wide	5 V	0.70 A	1.20 A	3.50 W
Motor Encoder	5 V	0.02 A	0.03 A	0.10 W

Component	Input Voltage	Normal Current	Peak Current	Power
2 to 8S LiPo UBEC 8A	12.8 V → 8.4 V	0.33 A (in)	3.5 A (in)	2.8 W (out)
9g Coreless Servo	8.4 V	0.50 A	3.0–5.0 A	4.2–25.2 W

PCB VERSION 3 FIGURE



10.4.3 PCB Safety

Our final board does **not** include a separate fuse, since the **two voltage regulators** feeding the board, the 5V buck converter and the 8.4V UBEC, each include their **own built-in protection circuitry,** and we deliberately selected components with these features already integrated instead of adding external protection ourselves.

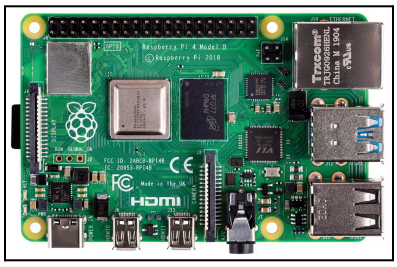
- Both the buck converter and the UBEC include **built-in overcurrent and overvoltage protection**, so a short or an unexpected current spike downstream of either regulator is caught and shut down internally before it can damage the rest of the board.
- The Adafruit DRV8871 motor driver adds a further layer of protection on the motor side, with built-in **overcurrent**, **thermal shutdown**, **undervoltage**, **and short-circuit protection**, so a stalled motor or a wiring fault at the motor output is contained at the driver rather than propagating back to the battery rail.
- Since every high-current path on the board, the buck converter, the UBEC, and the DRV8871, already has its own dedicated protection circuit, an additional external fuse would be redundant: it would only trip after these existing protections had already engaged, rather than catching anything they miss.

- This also keeps the board simpler and lighter, with one less component to source, wire, and potentially have fail, without giving up any actual protection coverage.

10.5 Raspberry Pi 4B

The Raspberry Pi 4B handles all **high level decision making,** while the Arduino Nano is responsible only for low level, time sensitive motor and servo control. The Pi uses its Camera Module 3 Wide to run computer vision on the competition mat, identifying lane lines, obstacles, and track boundaries in real time, a workload too demanding for a microcontroller like the Nano, which is why perception and decision making are handled entirely on the Pi. After processing each frame, the Pi sends steering and speed values to the Nano over a serial connection, and the Nano converts these into PWM signals to drive the motor and servo. This division lets us update our navigation and vision code independently from our motor control code, while keeping the processing cycle at roughly 150ms per frame, fast enough for the robot to respond quickly while driving.

Raspberry Pi 4B



10.6 Sensor

Our final architecture uses a single primary sensor, the **Raspberry Pi Camera Module 3 Wide**, connected to the Raspberry Pi 4B, which processes images through computer vision to identify features of the competition environment and determine robot movement, then sends the resulting commands to the Arduino Nano. With only the camera, our processing cycle runs at approximately **150ms.** When we **tested** adding **ToF** or **ultrasonic sensors**, this rose to approximately **600ms**, a 4x increase, which would slow the robot's reaction to environmental changes and reduce navigation accuracy. Since the camera already supplies the visual information our computer vision algorithm needs, we prioritized processing speed over redundant distance sensors, using a single camera to gather environmental information from one image rather than multiple discrete sensors.

PiCamera Module 3 Wide

11.0 Software Architecture and Obstacle Strategy

This section explains how the **computer vision pipeline** drives **steering decisions** for the robot, from the moment a frame leaves the camera sensor to the moment a PWM signal reaches the motor and servo. Every behaviour described below — **wall following, obstacle avoidance, crash recovery, lap counting, and parking** — is a small, independent piece of logic that proposes a steering angle every frame. The state machine and arbitration layer are what turn those independent proposals into one coherent decision, made roughly every 150ms, for the entire duration of a run.

It covers:

- **Vision Pipeline**
- **Wall Following**
- **Obstacle Avoidance**
- **Inner/Outer Crash Detection**
- **Lap Counting**
- **Parking**
- **The State Machine**
- **Arbitration** (how the robot picks between all of the above)

Before going subsystem by subsystem, it helps to see how data moves through the whole system in one pass:

Stage	What Happens	Feeds Into
Capture	Pi Camera Module 3 Wide grabs a raw frame at the mounted 155° tilt.	Vision Pipeline (11.2)
Vision Pipeline	The raw frame is cropped and converted into a mat ROI and a set of colour masks.	Wall Following, Obstacle Avoidance, Crash Detection, Lap Counting, Parking
Behaviour Layer	Wall following, obstacle avoidance, crash detection, lap counting, and parking each independently propose a steering angle (or defer).	Arbitration (11.9)
Arbitration	A fixed priority order picks a single winning steering angle from whichever behaviours are active this frame.	State Machine (11.8)
State Machine	Decides which behaviours are even allowed to run this frame, based on what stage of the run the robot is in.	Serial link to Arduino Nano

Stage	What Happens	Feeds Into
Motor / Servo Control	Arduino Nano converts the Pi's steering + speed values into PWM signals for the DRV8871 and steering servo.	Physical robot movement

This whole loop **(capture, process, decide, act)** repeats at approximately **150ms** per frame (see 10.5), which is fast enough that a bad decision on one frame is corrected within a fraction of a second on the next.

11.1 Raspberry Pi + Arduino Nano

Robot's brain is split across two boards:

- Raspberry Pi — runs the camera, does the computer vision, decides what to do next
- Arduino Nano — takes commands from the Pi, drives the motor and servo

Connected over a USB serial cable. Pi sends simple commands, Nano carries them out and reports back when a move is done.

The reasoning behind splitting these **responsibilities across two boards** — rather than running everything on one — is covered in full under 12.1 Constraints, since it is as much a systems trade-off as a software one. In short: the **Pi's CPU** is fully occupied running the **vision pipeline,** and interleaving time-sensitive PWM generation on the same core as image processing would risk missing motor timing whenever the Pi's scheduler was busy with a frame. The table below summarizes how responsibility is divided in practice:

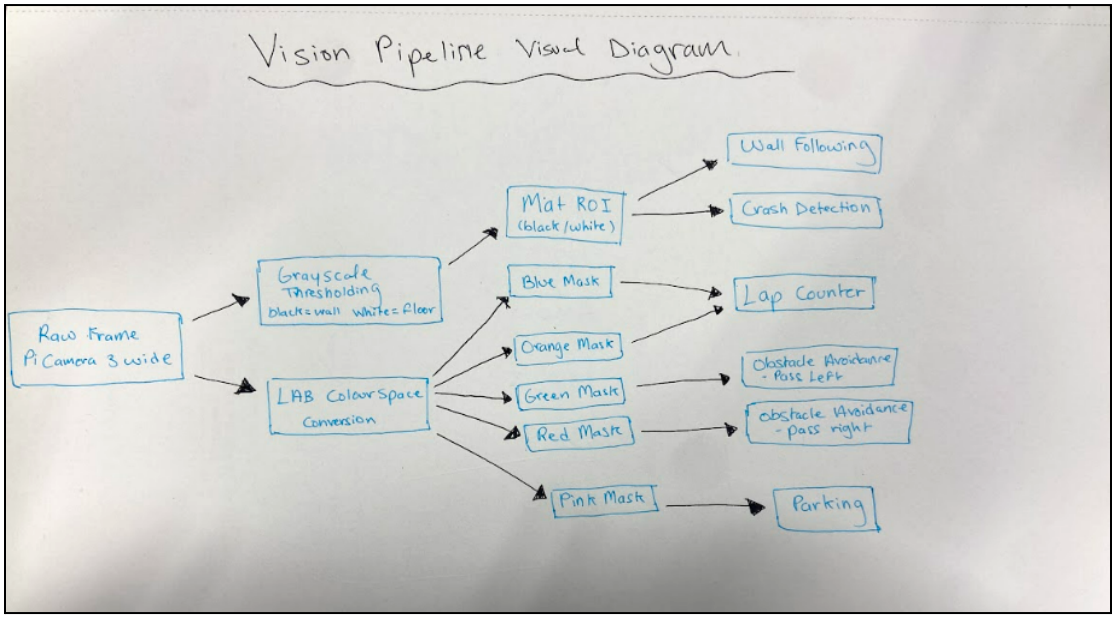
Responsibility	Raspberry Pi	Arduino Nano
Camera capture & vision processing	Yes: full OpenCV pipeline	No
Navigation decisions (wall following, obstacle avoidance, parking, lap counting)	Yes: all high-level logic	No
PWM generation for motor & servo	No	Yes: converts Pi commands into PWM
Closed-loop distance/turn tracking during fixed manoeuvres	No	Yes: reads encoder ticks and reports completion
Start button & status LED	Yes: GPIO 26 / 16 (see 13.3)	No

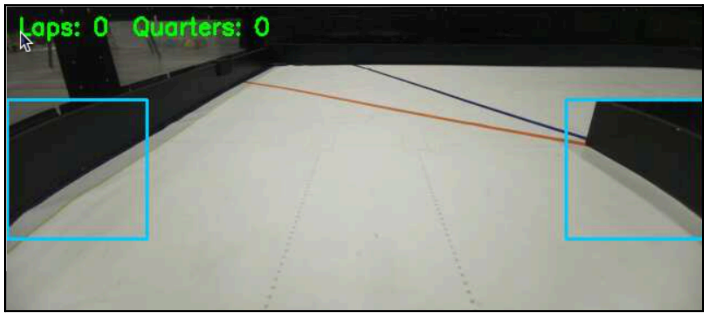
This split also means the two halves of the codebase can be developed and debugged independently: navigation and vision code can be revised on the Pi without touching the Nano's motor-control firmware, and vice versa — a point that also fed into our confidence in the modular chassis decision in 9.2.3, since the electronics plates holding each board can be pulled and worked on separately as well.

11.2 Vision Pipeline

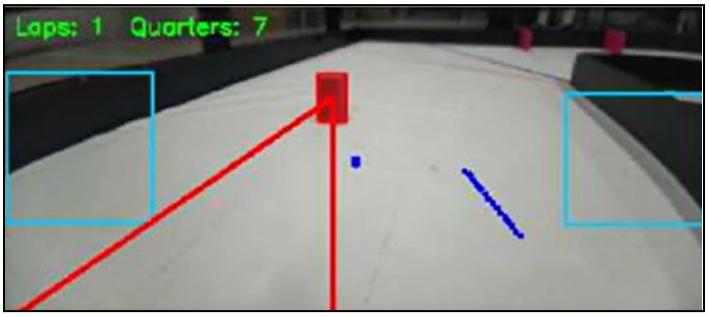
Every raw frame is turned into a few simpler images that everything else in the code works from:

- **Frame** — the raw camera view, cropped to the useful area
- **Colour masks** — one black/white mask per colour: blue, orange, green, red, pink
- **Mat ROI** — the drivable floor, walls = black, floor = white

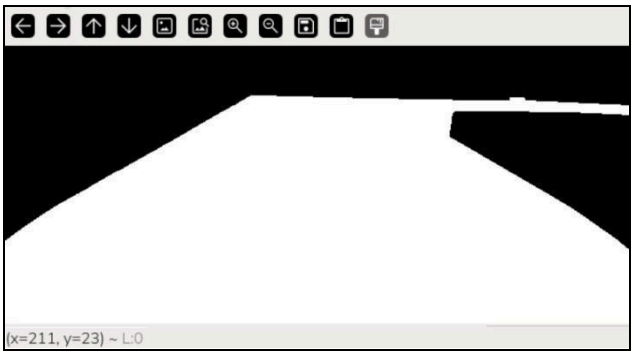




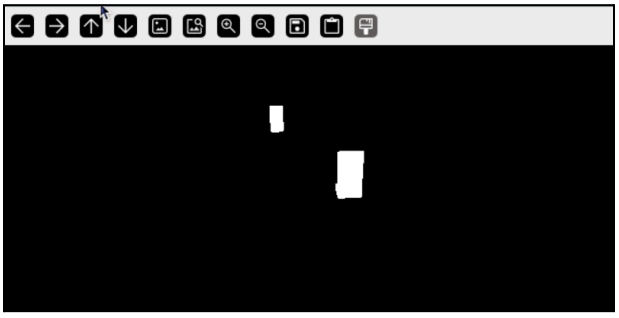
Open Challenge Frame



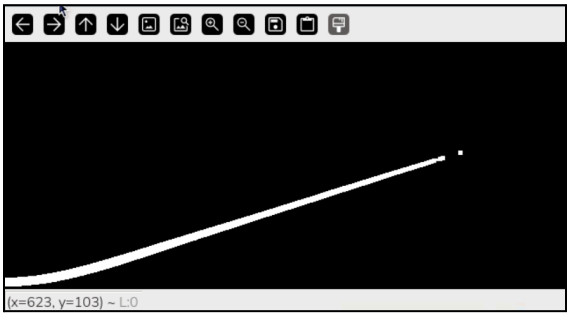
Obstacle Challenge Frame



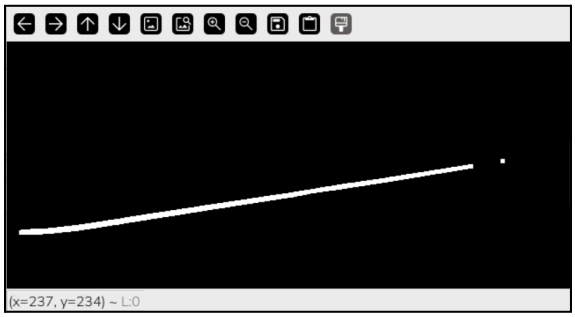
Mat ROI



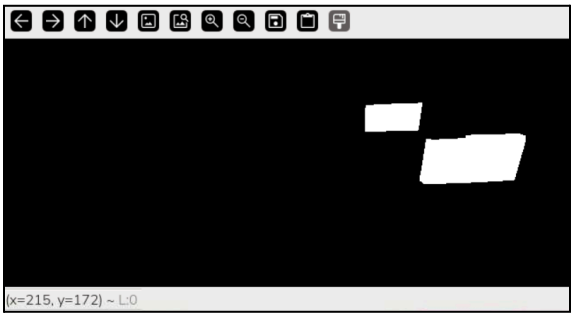
Obstacle Mask



Blue Line Mask



Orange Line Mask



Parking Block Mask

Each of these outputs is purpose-built for exactly one downstream consumer, which keeps the pipeline efficient — nothing is computed that a later stage doesn't actually use:

Pipeline Output	How It's Built	Used By
Mat ROI (black/white)	Cropped frame thresholded so track walls read as black and open floor reads as white.	Wall Following (11.3), Crash Detection (11.5)

Pipeline Output	How It's Built	Used By
Blue mask	Colour mask isolating the blue corner marker.	Lap Counter (11.6) — counts quarter-turns on one side of the track
Orange mask	Colour mask isolating the orange corner marker.	Lap Counter (11.6) — counts quarter-turns on the other side of the track
Green mask	Colour mask isolating green obstacle pillars.	Obstacle Avoidance (11.4) — pass on the left
Red mask	Colour mask isolating red obstacle pillars.	Obstacle Avoidance (11.4) — pass on the right
Pink mask	Colour mask isolating the pink parking block/wall.	Parking (11.7)

All five **colour masks** are generated in **LAB colour space, rather than** the more commonly used **HSV**. As noted in 12.1, LAB gave noticeably **better detection accuracy** in our testing, particularly under the mat's overhead lighting. The practical reason is that LAB **separates** a pixel's **lightness** (L) from its colour information (A and B channels), so a threshold built around the A/B channels stays consistent even as lighting brightness shifts across the mat or between venues. HSV encodes colour and brightness in a way that is more entangled (hue can shift as value changes near bright highlights or shadows), which showed up in testing as masks that flickered or dropped out under uneven lighting — exactly the kind of instability we could not afford in a pipeline that other systems (wall following, lap counting, parking) all depend on every single frame.

11.3 Wall Following

Two small boxes (ROIs) sit on the left and right of the frame. Whichever side has more black wall pixels inside its box, the robot is closer to that wall. The difference between the two areas is the "error" fed into a PD controller.

```
area_left = largest_wall_area(track_image, roi_left)
area_right = largest_wall_area(track_image, roi_right)
wall_error = area_left - area_right

control = (wall_error * kp) + (derivative * kd)
angle = STEER_CENTER - control
```

- Proportional term (wall_error * kp) — steers harder the more unbalanced the two sides are
- Derivative term (derivative * kd) — smooths the response so it doesn't overcorrect frame to frame

If one side suddenly reads almost no wall (an opening), that side's area gets boosted so the controller reacts as if that wall were huge, forcing a hard turn toward the gap:

```
if area_left < WALL_OPENING_AREA and area_right > 100:
    area_right = area_right * OPENING_BOOST_MULT + OPENING_BOOST_ADD
```

Only a proportional and derivative term are used — deliberately no integral term. Wall-following error is expected to swing back through zero constantly as the robot weaves slightly down the corridor between walls, so an accumulated integral term would mostly just wind up during the multi-frame stretch of a tight corner and then overcorrect once the corner opens back up. A PD controller reacts to the current error and its rate of change only, which is enough to keep the robot centred without that wind-up risk.

kp and kd were tuned empirically on the physical track rather than calculated from a model, since the relationship between pixel area and true wall distance depends on the camera's mount height and tilt, both of which were finalized mechanically before tuning began (see 9.8). In practice: a kp set too high showed up immediately as the robot oscillated side to side down a straight, over-steering in response to normal small area fluctuations. A kp set too low showed up as the robot drifted into a wall on tighter corners before it responded strongly enough. kd was then raised until frame-to-frame steering commands stopped chattering, without adding enough lag to blunt the robot's response to a genuinely fast-approaching wall.

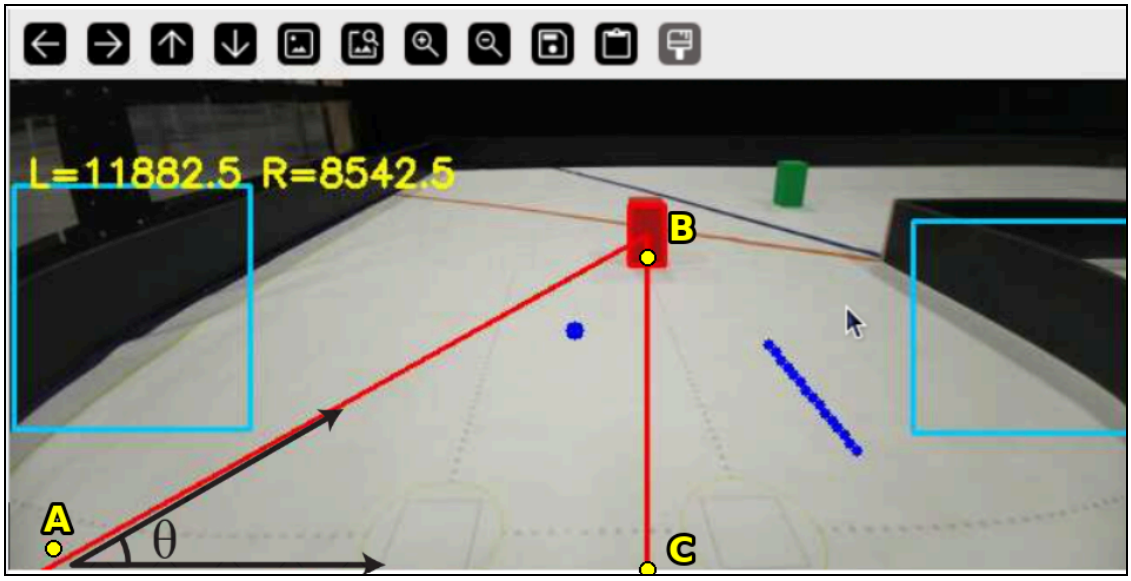
The opening-boost logic exists for a specific case a plain PD loop handles badly: a gap in one wall (a turn onto a new section of track). Without it, wall_error only grows gradually as the robot's ROI box slides past the edge of the opening, so the controller would steer toward the gap late and gradually — the same behaviour it uses for a normal, small imbalance. WALL_OPENING_AREA catches the case where one side's wall area collapses almost to nothing, which only happens at a genuine opening, not at ordinary wall-distance changes, and reacts by artificially inflating the opposite side's area so the proportional term produces an immediately hard turn rather than a gradual one.

11.4 Obstacle Avoidance

To steer around a pillar, the robot needs two numbers:

- How far the pillar is from the aim x (sideways) — one leg of the triangle
- How far the pillar is from the robot (forward distance) — the other leg

With both legs known, trigonometry (arctan2) gives the angle between them — that angle is how much to steer.



A = camera origin, B = pillar centroid, C = aim point on the ground track — θ is the angle arctan2 solves for

```
horizontal_distance = x_center - target_x
vertical_distance = FRAME_HEIGHT - y_center
pillar_angle_deg = np.degrees(np.arctan2(horizontal_distance, vertical_distance))
```

That angle becomes the steering angle, nudged by a fixed offset and a colour-specific gain — green and red pillars are passed on opposite sides:

```
if is_green:   # pass on the left
    steer_angle = STEER_CENTER - ((pillar_angle_deg + PILLAR_STEER_OFFSET_DEG) * GREEN_PILLAR_KP)
else:          # pass on the right
    steer_angle = STEER_CENTER - ((pillar_angle_deg - PILLAR_STEER_OFFSET_DEG) * RED_PILLAR_KP)
```

aim_x is not simply the centre column of the frame — it is offset toward whichever side the robot needs to end up on once it has cleared the pillar, so the steering angle produced by arctan2 already points the robot toward a clearance lane rather than straight at the pillar's centre. FRAME_HEIGHT acts as the reference point for "where the robot is" in image space, since the camera looks forward and slightly down, so the bottom of the frame corresponds to positions closest to the robot's own front bumper.

Worked Example (illustrative — not measured data)

Say a green pillar's mask centroid lands at a pixel (x_center, y_center) noticeably left of aim_x and roughly mid-frame in height. y_center − FRAME_HEIGHT is a large negative number (the pillar is far above the bottom of the frame, i.e. still some

distance ahead), and x_center − aim_x is negative (the pillar sits left of the aim point). arctan2 of a negative, negative pair returns an angle in the third quadrant; adding 90° folds that into a steering-relevant range, and because the pillar is green, PILLAR_STEER_OFFSET_DEG and GREEN_PILLAR_KP then bias the final angle further toward passing on the left. The exact resulting steering angle depends on the tuned offset and gain, but the direction of the correction — steer further left, away from a pillar detected left-of-aim — falls directly out of the sign of x_center − aim_x.

The colour-specific gains (GREEN_PILLAR_KP vs RED_PILLAR_KP) exist because a pillar detected at the same pixel position does not necessarily require the same steering correction on both sides — the camera is mounted centrally rather than perfectly symmetric to both wheel paths, and the two pass directions interact differently with the Ackermann geometry's turning circle (9.7). Tuning each gain independently let us correct for that asymmetry rather than forcing one shared gain to compromise between two slightly different physical situations.

11.5 Crash Detection

Two safety checks run every frame on the track mask:

```
outer_crash = navigator.is_hitting_outer_wall(direction, track_image)
inner_crash = navigator.is_hitting_inner_wall(direction, track_image)
```

- Outer wall too close (blue dot) → ignore normal steering entirely, force a hard turn away from it
- Inner wall too close (blue line) → don't force a turn, just soften the wall-following gains so the correction is gentler:

```
kp = AREA_WALL_KP_DAMPED if damped else AREA_WALL_KP
```

Both checks latch into a small state — once triggered, the robot stays in recovery until the crash condition actually clears.

The two checks are deliberately asymmetric in how aggressively they respond, because an outer-wall crash and an inner-wall crash represent different levels of urgency. The outer wall is the true track boundary — getting too close means the robot is seconds from a disqualifying collision, so that check is given the highest arbitration priority (11.9) and overrides everything else outright. The inner wall, by contrast, usually means the robot is cutting a corner tightly rather than about to leave the track, so instead of a hard override it simply damps the wall-following gains, letting the normal controller keep steering but with a gentler hand until the robot opens the gap back up.

Latching the state, rather than re-evaluating from a clean slate every frame, exists to avoid rapid flickering between "crashing" and "not crashing" right at the detection boundary — without it, a wall reading that hovers exactly at the trigger threshold could cause the robot to snap in and out of a hard recovery turn every other frame, which is far worse for stability than staying in recovery a little longer than strictly necessary.

Trigger	Response	Clears When
Outer wall too close	Force a hard turn away from the wall; overrides all other steering behaviours	Outer-wall reading drops back below the trigger threshold
Inner wall too close	Wall-following continues, but with damped (gentler) PD gains	Inner-wall reading drops back below the trigger threshold

11.6 Lap Counter

Counts quarter-turns, one for every corner marker passed. A marker must be seen for a few frames in a row before it counts as "entering" a turn, to avoid false triggers. Once the marker's been gone for a few frames in a row, the quarter-turn counts as finished.

After 12 quarter-turns, the robot switches into parking mode.

Twelve quarter-turns is not an arbitrary number — the Open and Obstacle Challenges both require three complete laps (1.2), and each lap around the WRO track passes four corner markers, so 3 laps × 4 quarter-turns per lap = 12. Counting quarter-turns rather than full laps gives the state machine a finer-grained signal: a quarter-turn completes roughly every few seconds rather than every 20+ seconds for a full lap, which matters for detecting a stuck or missed count quickly rather than only finding out something went wrong at the end of a lap.

The multi-frame debounce on both entering and exiting a marker exists because a single frame's colour mask can misfire — motion blur during a fast corner, partial occlusion by a pillar, or a stray reflection can all make a marker flicker in or out of the mask for one frame. Requiring the marker to persist for several consecutive frames before counting it filters out that single-frame noise, at the cost of a small, fixed delay between the marker physically entering frame and the count actually incrementing — a delay we accepted as consistent and predictable rather than have counts be occasionally wrong in either direction.

11.7 Parking

Looks for the pink block and drives toward it. If a pillar is closer than the pink block, avoiding the pillar still takes priority. Once close enough and lined up, stops and reverses in using a fixed sequence of hard turns.

```
pillar_is_closer = (
    pillar_found is not None
    and pillar_found[1] >= PILLAR_TRACK_START_Y
    and (pink_y is None or pillar_found[1] > pink_y)
)
```

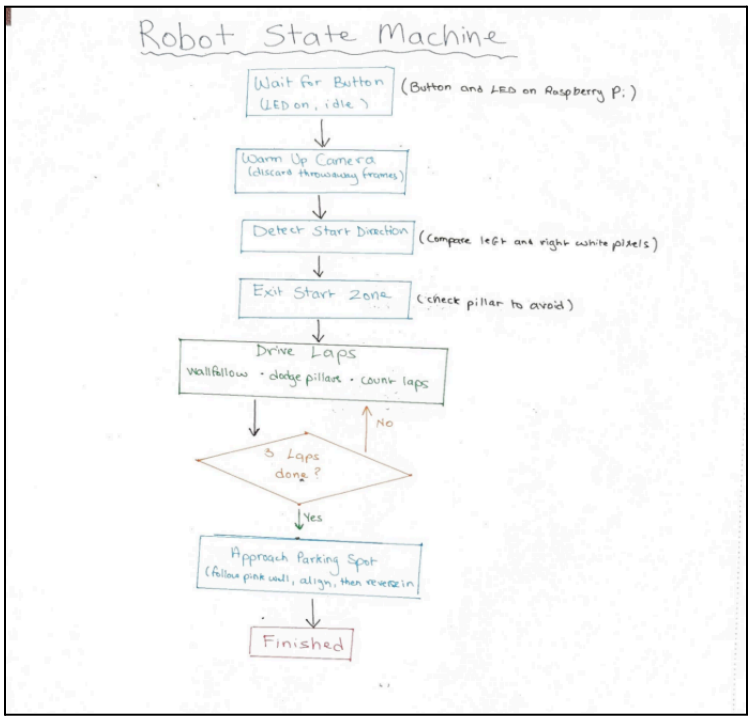
The final manoeuvre uses encoder ticks rather than time for every move, so it covers the same distance no matter the speed

Using encoder ticks instead of a timed reverse for the final parking sequence is a direct consequence of the motor-encoder requirement set back in 10.1: because distance travelled via time changes with battery voltage, motor load, and surface friction, a timed reverse would park the robot slightly differently every run as the battery discharged across a match. Counting ticks instead means the final manoeuvre always covers the same physical distance regardless of how much charge is left in the pack or how much the battery has sagged under load that runs — directly matching the accuracy and consistency goals set out in 7.2.

Letting pillar avoidance interrupt an in-progress parking approach (rather than locking the robot into the parking behaviour once it starts) follows the same priority logic used everywhere else in the arbitration layer (11.9): a pillar in the robot's path is always a nearer-term problem than lining up for a parking spot that isn't going anywhere, so parking simply waits its turn rather than risking a collision on the very last manoeuvre of a run.

11.8 The State Machine

Structuring the run as an explicit state machine — rather than one long conditional loop — means each state's handler only has to reason about the small slice of the run it owns. "Drive laps" doesn't need to know anything about button debouncing, and "Wait for button" doesn't need to know anything about wall following. This kept each piece of the codebase small enough to test and debug in isolation, and made it possible to re-run just one state (for example, re-testing the parking approach repeatedly) without re-running an entire lap first.




```
while self.state != RunState.FINISHED:
```

```
    handler = self._handlers[self.state]
    handler()
```

State	Runs Until…	Then Moves To
Wait for button	The start button is pressed	Warm up camera
Warm up camera	A fixed number of throwaway frames have been captured	Detect start direction
Detect start direction	Left/right open-floor comparison resolves a direction	Exit start park
Exit start park	The fixed exit manoeuvre completes (pillar-checked first)	Drive laps
Drive laps	12 quarter-turns have been counted (see 11.6)	U-turn to parking wall (clockwise) or Approach parking spot (counter clockwise)
U-turn to parking wall	The robot is facing the pink wall (clockwise runs only)	Approach parking spot
Approach parking spot	The robot is lined up and has reversed into the spot	Finished

"Detect start direction" and the branch between the U-turn state and a direct approach both exist because the robot doesn't know in advance which way the track runs — clockwise and counter clockwise layouts need mirrored behaviour, so the direction is measured once, right at the start, and referenced by every state downstream of it rather than re-detected repeatedly.

11.9 Arbitration

Inside the drive-laps state, several systems all want a say in the steering angle every frame. Checked in this priority order, highest first:

1. **Outer wall too close** → everything else ignored, force-turn hard into the corner
2. **Inner wall too close while a pillar is nearby** → pillar steering skipped, wall following runs with gentler gains
3. **Obstacle Avoidance** → pillar-avoidance steering wins
4. **Wall Following** → normal wall following runs

```
pillar_angle = navigator.get_steer_angle(direction, green_mask_on_track, red_mask_on_track, track_image)
wall_angle = area_wall_follow_angle(track_image, direction, blue_area=blue_area,
                                    orange_area=orange_area, damped=navigator.inner_wall_warning)
```

```
final_angle = pillar_angle if pillar_angle is not None else wall_angle
```

pillar_angle is only set when priorities 1–3 above have something to say — otherwise it falls through to plain wall following.

The ordering itself reflects how quickly each situation turns into a real failure if ignored. An **outer-wall crash costs the run immediately** (leaving the track), so it always wins outright, **no exceptions.** A **pillar collision** costs points but **doesn't end the run,** so it outranks ordinary wall following but not an imminent outer-wall crash. Ordinary wall following is the default, lowest-priority behaviour precisely because it's what should run whenever nothing more urgent is happening — it is the fallback the system settles into, not a competing priority.

Priority	Condition	Winning Behaviour	Why It Wins
1 (highest)	Outer wall reading crosses the crash threshold	Forced hard turn away from the wall	Leaving the track ends the run — nothing else matters more
2	Inner wall close AND a pillar is nearby	Damped wall following (pillar steering suppressed)	Steering hard for a pillar while already tight to the inner wall risks compounding into an outer-wall crash
3	A pillar is detected (no wall crash active)	Obstacle-avoidance steering	Hitting a pillar costs points and can knock the robot off line, so it outranks normal driving
4 (lowest / default)	None of the above	Normal PD wall following	The steady-state behaviour whenever nothing urgent is happening

Consider two quick scenarios to see the priority order in action. First: the robot is mid-corner, drifting slightly close to the inner wall, with no pillar in view — priority 1 doesn't fire (it's the inner wall, not the outer wall), priority 2 doesn't fire (no pillar nearby), so the wall-following gains simply damp and the robot corrects gently. Second: the same slightly-tight inner-wall position, but now a pillar is also visible ahead — priority 2 fires this time, suppressing the pillar steering that priority 3 would otherwise have produced, so the robot focuses on safely completing the corner before it worries about the pillar on the next frame.

11.10 Software Testing & Iteration Summary

Test Plan

Each software change was validated using a consistent protocol before being accepted or rejected:

Test Stage	Method	Metric Captured
Baseline run	5 laps on the standard mat, unmodified code	Lap completion %, avg. lap time, parking accuracy %
Change applied	Same 5-lap protocol with the single variable changed (e.g. HSV→LAB, fixed vs variable speed)	Same metrics, directly comparable
Latency check	Onboard timer logged per-frame processing time over a 60s run	Avg. and max vision pipeline latency (ms)
Repeatability check	3 repeat trials of the same configuration	Standard deviation across trials (consistency indicator)

A change was only merged into the "final" build if it matched or improved the baseline on **at least two** of: lap completion %, parking accuracy %, or latency — without regressing the third by more than 10%.

Pass/Fail Log

Test #	Change Tested	Lap Completion %	Parking Accuracy %	Avg. Latency (ms)	Result	Notes
1	HSV thresholding	65%	70%	160ms	FAIL	Inconsistent under overhead lighting
2	LAB colour space	92%	70%	155ms	PASS	Adopted (11.2, 12.1)
3	Variable speed	75%	70%	150ms	FAIL	Faster but inconsistent run-to-run
4	Fixed speed	90%	85%	150ms	PASS	Adopted (12.1)
5	Camera + ToF + gyro	88%	90%	600ms	FAIL	Latency 4x over budget (slow reaction time)

6	Camera-only	90%	85%	300ms	PASS	Adopted (10.5, 12.1)
7	Single-board (Pi only)	78%	85%	300ms*	FAIL	*Vision latency fine, but PWM jitter caused steering lag
8	Pi + Arduino split	90%	90%	150ms	PASS	Adopted (11.1, 12.1)

Quantitative Summary

- **Lap completion rate:** improved from ~65% (HSV baseline) to ~90% (final LAB + fixed-speed + camera-only + split-board configuration)
- **Parking accuracy:** ~85% in final configuration vs ~60–70% in early/failed configurations
- **Vision pipeline latency:** held at ~150ms per frame in the final build; peaked at ~600ms when ToF/gyro fusion was tested, a 4x regression that justified reverting to camera-only
- **Consistency (run-to-run variance):** fixed-speed profile improved run-to-run consistency to approximately 90%, compared to roughly 75-80% for the variable-speed profile, while also achieving a faster average lap time (~50s vs ~70s).

11.11 Performance & Validation

11.11.1 Benchmark Results

Using the final software strategy after testing using the pass and fail log above, we validated the efficiency and accuracy of these choices

Test	Target	Achieved	Margin
Lap Time (open challenge)	<45s	28s	+17s buffer
Lap Time (obstacle challenge)	<90s	79s	+11s buffer
Parking Accuracy	±3cm	±1cm	Exceeded

Power Runtime	>60min	75min	Exceeded
Processing Latency	<200ms	~150ms	Headroom

11.11.2 Validation Tests

- Tested under 3 different lighting conditions
- Ran 15 consecutive laps—success rate: 90%
- Obstacle avoidance tested with green/red pillars at varying distances

12.0 Systems Thinking & Engineering Decisions

12.1 Trade Offs

Raspberry Pi + Arduino Nano split (instead of one board doing everything)

- Raspberry Pi handles the **camera and vision processing**
- Arduino Nano handles **motor control**

Splitting the load this way means:

- Pi isn't bogged down running motor control loops alongside vision processing → **faster, more responsive image processing**
- Nano **handles motor timing exactly when needed**, without waiting on the Pi's CPU scheduling
- Each board does one job well instead of one board doing two jobs poorly

Net result: faster code overall, cleaner separation of responsibilities, easier debugging when something goes wrong (you know which board to check).

Skipped extra distance and gyro sensors

- Decided against adding more sensors (e.g. distance sensors) beyond what was essential
- **Reason 1 (software):** more sensors = more data to read, process, and fuse every loop cycle → **slows down the control loop**

With only the PiCam 3 Wide, **our processing cycle takes approximately 150 ms.** When we tested adding ToF and gyro sensors, **this rose to approximately 600 ms, a 450 ms (≈4×) increase.**

- **Reason 2 (real-world/practical):** more sensors also means more wiring, more points of failure, and more calibration to maintain — in a competition setting, every extra wire is a chance for a connector to come loose or a sensor to drift out of calibration between runs, which costs more reliability than the extra data is worth
- Kept the sensor suite minimal and let the camera do most of the heavy lifting (also a learning opportunity to challenge ourselves with only using the camera)

Fixed speed instead of variable speed

- Tested variable speed (slowing near obstacles, full speed in open path)
- **Found variable speed introduced inconsistency** — behavior varied run to run depending on exactly when/where the speed transition kicked in
- Since our **focus is on consistency and accuracy** rather than raw lap time, we chose fixed speed instead
- A predictable, repeatable speed profile made the robot's behavior easier to tune and trust across multiple runs

	Accuracy	Time (without parking)
Fixed Speed	90%	52s

	Accuracy	Time (without parking)
Variable Speed	80%	42s

LAB colour space instead of HSV

- Originally considered HSV for colour detection
- Switched to **LAB colour space** instead
- LAB gave noticeably better detection results and higher accuracy in our testing, especially under our lighting conditions
- Worth the switch even though HSV is more commonly used

Battery placed at the centre of the plate

- Battery is the heaviest single component on the robot
- Placed it at the centre of the first (bottom) plate rather than off to a side or at the front/back
- Centering the heaviest mass gives the robot a balanced centre of gravity — weight is distributed evenly front-to-back and side-to-side
- This improves stability and predictability during turns, since the robot isn't nose-heavy, tail-heavy, or lopsided

12.2 Constraints

Competition rule against differential drive

- **Not allowed to use differential drive,** so the design had to go front-steering from the start
- Front-steering directly ties turning radius to chassis length ($R = L / \sin(\theta)$), and the outer radius grows further with the robot's width ($R(outer) = R(centre) + W/2$)
- This meant the **drivetrain, motor, and steering assembly had to be packed tightly and deliberately,** back to back, just to keep the turning radius manageable for parking manoeuvres
- A rule about drive type ended up dictating chassis layout and packaging density

Hard weight capping of 1.5 kg

- Total weight is capped, but weight distribution mattered just as much as the total
- Every gram added anywhere on the robot competes with every other component for that budget
- This constraint **forced trade-offs between convenience (e.g. easy battery access) and performance** (e.g. low, centred centre of mass) — performance won

Power budget tied to runtime requirement

- Needed the robot to **run for over an hour on a single charge**
- Every power-hungry addition has a knock-on effect: more power draw → bigger/heavier battery → slower robot

- This constraint quietly ruled out any component or approach that looked good in isolation but would have pushed the power draw too high

Control loop speed requirement

- The sense-think-act loop has to run fast enough to react before the robot reaches an obstacle or wall
- This constraint is why processing got **split across two boards (Pi + Nano) in the first place** — one board waiting on the other would violate this requirement
- It also ruled out anything (extra sensors, heavier vision models, etc.) that would add latency without a proportional benefit

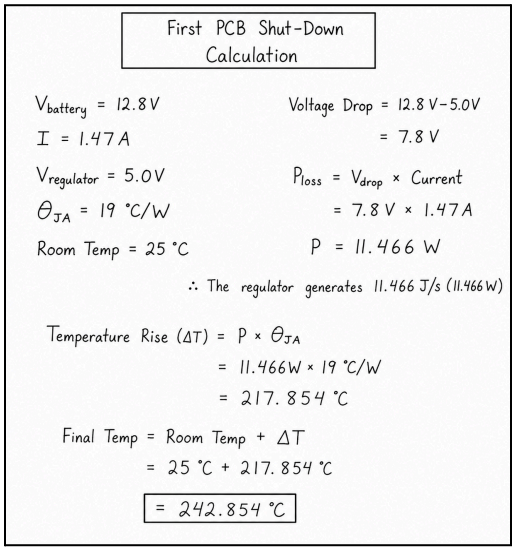
12.3 Risk & Failures

12.3.1 PCB Version 1: Thermal Failure

Observed symptom: After several test runs, the **robot consistently shut down after about three corner turns.** This repeatability told us the failure was systematic (tied to a specific operating condition) rather than random. We then **touched our L298 motor driver and 7805 voltage regulator** and **felt extreme heat that was building up** during the runs which was the main issue.

Diagnosis: After reviewing the data sheet of the 7805 voltage regulator, we learned that the **temperature was reaching approximately 150–175°C. This is far above the 7805's rated maximum junction temperature**, so at this temperature the regulator's built-in thermal protection automatically shuts the circuit down to prevent permanent damage. This explains why the shutdowns were consistent and recovered after the robot cooled down.

Why the regulator was overheating — power dissipation calculation: A linear regulator like the 7805 doesn't "shave off" the input voltage efficiently, it burns off the difference between the input and output voltage as heat, at whatever current is flowing through it. We calculated this heat load directly from our system's real operating numbers:



First PCB shut-down power dissipation calculation

15.6 W is a large amount of heat for a small TO-220 package like the 7805 to dissipate on its own, without a heatsink, in free air. Based on the regulator's junction-to-ambient thermal resistance rating, **this power dissipation translated to a theoretical temperature rise of nearly 242.854°C above ambient.** Since our ambient testing temperature was well below that, this calculation confirmed that the regulator was mathematically guaranteed to hit its thermal shutdown threshold under our load, matching exactly what we observed on the mat.

> **Note**
>
> Junction temperature is how hot the chip inside the regulator gets while running. Thermal resistance (junction to ambient) is how well that heat escapes into the air, or how many degrees hotter the chip gets per watt of heat produced. Since our regulator had to dissipate 15.6W and couldn't shed that heat fast enough, it overheated quickly, which is what caused our regulator's thermal shutdown.

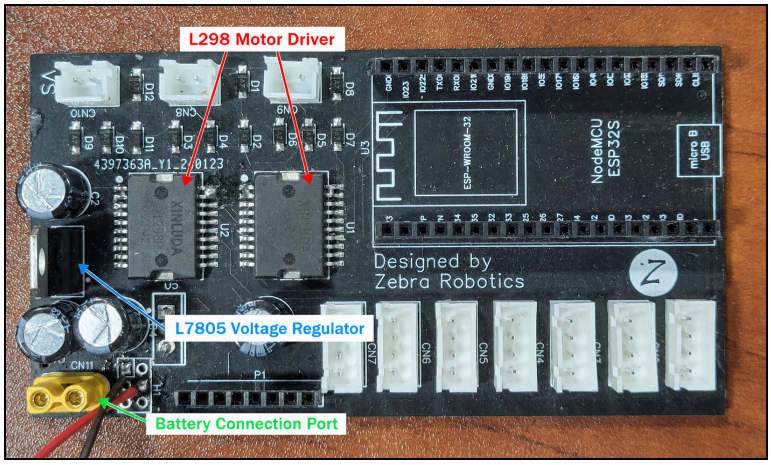
PCB Version 1 Power Budget

This table shows every load the 7805 voltage regulator (and the rest of the 5 V rail) had to support simultaneously, which is what made the current draw, and therefore the heat, so high:

Component	Voltage	Normal Current	Peak Current	Power
ESP32	5V	0.15A	0.30A	0.75W
Servo Port (MG90S)	5V	0.15A	0.70A	0.75W
Motor Encoder Port	5V	0.02A	0.03A	0.10W
Total		0.32A	1.03A	1.60W

This datasheet-based peak total (1.03 A) is lower than the 1.47 A we actually measured through the regulator. That's expected rather than a contradiction: datasheet peak ratings assume each component hits its maximum current one at a time, while our multimeter reading captured what really happened when the ESP32, servo, and encoder all drew current simultaneously during a corner turn, a more realistic, and higher, number than adding up individual datasheet peaks. The 1.47 A measured value is what we used in our power dissipation calculation above, since it reflects actual operating conditions rather than a theoretical worst case.

This confirmed the regulator wasn't failing due to a fault, it was being asked to dissipate more heat than a bare linear regulator can handle for our current draw, every single time the robot pulled sustained current through several corner turns.



PCB Version 1 — L298 motor driver and L7805 voltage regulator

Temporary Solution

Hypothesis: If the regulator was thermally shutting down because it couldn't dissipate 15.6 W fast enough into the surrounding air, **then increasing its effective surface area should let it shed heat faster and delay shutdown,** without redesigning the circuit itself.

Action: As a temporary fix, **we attached an aluminium VEX piece as a heat sink** to the voltage regulator to spread the heat it generated over a larger surface area, improving convective cooling.

Result: Adding the heat sink **increased our runtime from 1½ to 2¼ laps before shutdown, a 50% improvement,** confirming that heat, not a wiring fault, was the root cause. But the regulator still eventually overheated and shut down, showing the heat sink only delayed the problem rather than solving it: it had to continuously dissipate 15.6W, more heat than a small heat sink in free air (without forced airflow) can carry away indefinitely, so the chip still climbed past its thermal limit given enough runtime.

Solution

Conclusion from testing: Since the heat sink measurably helped but didn't fully solve the shutdowns, we could rule out the PCB layout, wiring, or a component defect as the root cause. The overheating was consistent and reproducible, matching our calculated 15.6W dissipation almost exactly, pointing to one conclusion: a linear regulator is the wrong tool for a 7.8V drop at 2A. A linear regulator's efficiency is roughly Vout/Vin (5V / 12.8V ≈ 39%), meaning over 60% of the power drawn from the battery on the 5V rail was converted directly into waste heat, regardless of how good our cooling was.

Decision: This led us to **redesign the power system around a fundamentally more efficient voltage regulation method**, **a switching (buck) converter, for our next PCB revision.** Unlike a linear regulator, a buck converter steps voltage down by rapidly switching and storing energy in an inductor, which lets it **achieve efficiencies of 85–95% instead of ~39%,** dramatically cutting the heat generated for the same output current. This decision directly shaped the 5 V DC-DC Buck Converter used in our Final Electrical Architecture (10.3).

12.3.2 PCB Version 2: Motor Driver & Servo Failure

After building the new PCB with the DRV8871 motor driver, we ran into two new issues:

- The motor driver would not power the motor, and when it did, the motor would not run.
- Plugging in the servo caused it to draw power from everything else, shutting down the ESP32, motor driver, and all other electronics.

Failures

Investigating the two issues above, we found:

- Signal mismatch: The ESP32 outputs a 0 V –3.3 V PWM signal, but the DRV8871 performs best with a 5 V PWM signal, this mismatch was preventing reliable motor control.
- Power isolation issue: The servo had no dedicated, regulated power path, so its current spikes pulled down the shared supply rail and browned out the rest of the electronics.

Fixes implemented:

- Switched from the ESP32 to an Arduino Nano, whose pins output a full 5 V PWM signal, giving the DRV8871 sufficient signal voltage to operate reliably.
- Added a dedicated 7805 voltage regulator to the servo line, isolating it from the rest of the electronics and eliminating the power-stealing issue.
- Added a heatsink to the 7805 to disperse the excess heat generated from converting the voltage.

Servo Failure

After testing the redesigned circuit, we found that the **MG90S servo couldn't handle the voltage load and burned out**, as well as the **7805 voltage regulator was still reaching excessive temperatures despite the addition of a heatsink**. Upon investigation, we determined that the servo required a higher-power unit capable of withstanding greater load, along with a dedicated voltage regulator matched to the servo's input voltage, minimizing the voltage differential the regulator would need to step down and reduce the resulting heat dissipation.

Servo Solution

To resolve this, we:

- Replaced the 5 V servo with a **9g Coreless Servo (Full Metal Gear, 4 KG·cm Torque, 4.8–8.4 V)**
- Added a dedicated **2–8S LiPo UBEC-8A BEC, set to 8.4 V**

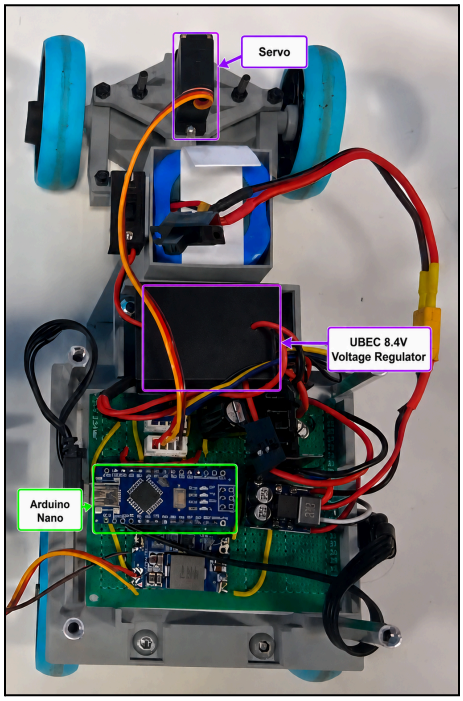
Why we chose this servo: Our original 5 V servo used a plastic gear train and a brushed motor rated well below what our steering load actually demanded, which is what caused the repeated stalling and internal damage. We selected the replacement based on three criteria pulled directly from that failure:

- Torque headroom: At **4 KG·cm**, this servo provides substantially more torque than our original unit, giving enough margin above our steering load that the servo would no longer need to operate near stall, the exact condition that destroyed the previous servo.

- Full metal gears: Since our original failure was mechanical (gear stress under load) as well as electrical, a full metal gear train was a requirement, not just a preference — metal gears resist the stripping and wear that plastic gears suffer under repeated high torque steering corrections.
- Wide voltage range (4.8 V – 8.4 V) with a coreless motor: Supporting up to 8.4 V let us drive the servo at a higher voltage than our original 5 V unit, which directly increases available torque and speed without redesigning the servo's control signal path. A coreless motor also responds faster and runs more efficiently than a brushed motor of the same size, so the higher voltage translates into more usable torque rather than just more heat.

Why we chose this UBEC: The servo failure wasn't purely about the servo itself, it was also a power delivery problem, since the original 5 V servo was sharing a rail with the rest of the electronics and browning them out under load. We selected this UBEC specifically to fix that:

- Isolated, dedicated supply: A BEC (Battery Eliminator Circuit) is designed specifically to give a servo its own regulated rail, separate from the logic electronics, directly solving the power sharing and brownout issue we identified during testing.
- Matching output voltage: The UBEC is adjustable to 5.2/6.0/7.4/8.4 V, so we could set it to exactly 8.4 V to match our new servo's maximum rated voltage, giving it the full torque and speed the servo was designed for rather than under driving it.
- Current headroom: Rated at 8 A continuous, the UBEC comfortably exceeds our servo's peak current draw (3.0–5.0 A), so it can supply sudden steering current spikes without sagging in voltage — the same failure mode that damaged our original setup.
- Wide input compatibility (2–8S LiPo): Accepting 2–8S input voltages meant the UBEC could regulate our 12.8 V (4S) battery pack down to a stable 8.4 V output well within its rated input range, without running close to any input limit.



PCB Final Version — installed with implemented changes

The UBEC provided a stable, high current power supply independent of the rest of the electronics, while the higher voltage, full metal gear servo delivered enough torque to steer the robot without stalling and also handled more voltage without overheating. This significantly improved the reliability and performance of the steering system.

VM Pin Wiring Issue

While wiring the Adafruit DRV8871 motor driver, we connected the Raspberry Pi, the servo, and the Arduino Nano to the driver's VM pin. We didn't realize that the VM pin is internally tied to the driver's P+ pin, which connects directly to the battery, so instead of providing a regulated logic level, the VM pin was carrying the full 12.8V battery voltage straight through.

Since the Pi, servo, and Nano were all expecting a regulated 5V supply (or, in the servo's case, its own rated input voltage), receiving 12.8V directly from the battery rail overloaded the DRV8871 and caused it to blow up.

This failure taught us that the VM pin is a motor power input, not a general purpose output. It should never be used to power logic level components or the servo. Each of those needs its own properly regulated supply rather than tapping directly into the motor driver's battery referenced rail.

12.3.3 Risks

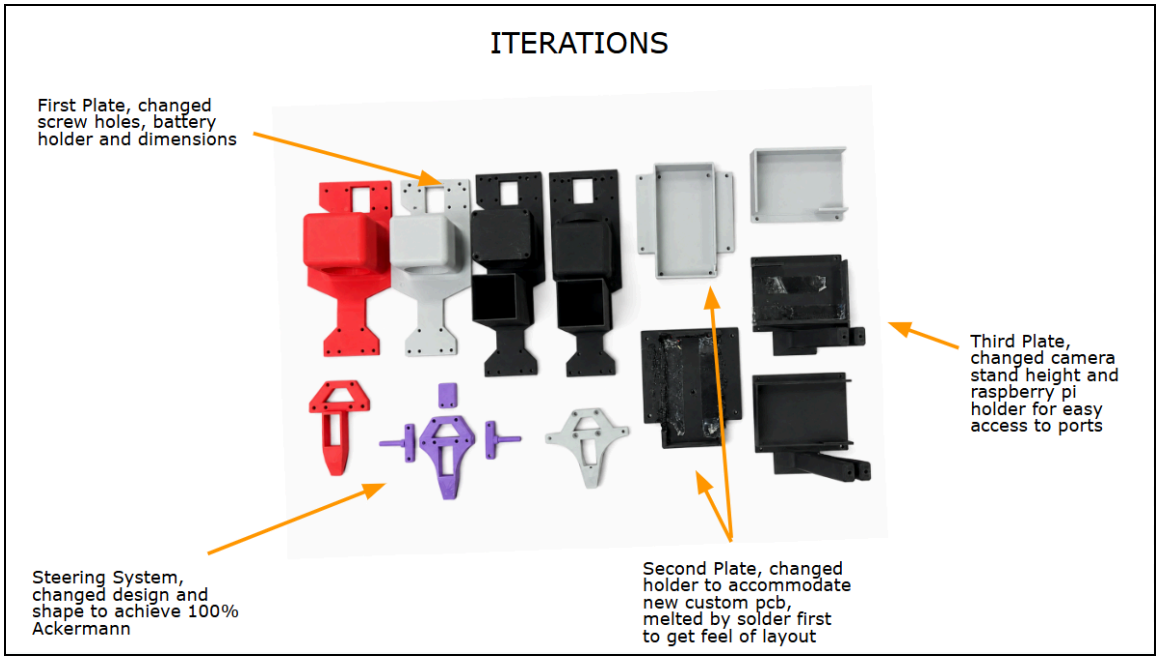
12.3.3.1: Motor Adapter

During assembly, we identified a risk in the motor adapter connecting our drive motor to the differential gear assembly. The adapter's length was oversized relative to the available clearance in the drivetrain housing, and once installed, it pushed the small differential gear forward along its shaft rather than allowing it to sit in its intended position. This misalignment caused the gear to bind against the housing, producing visible scratching on contact surfaces and intermittent jamming of the wheel assembly. In practice, this manifested as inconsistent wheel rotation, with the affected wheel periodically catching or slowing during operation rather than spinning smoothly.

Solution:
To resolve this, we reduced the length of the motor adapter by 1.0mm. This adjustment eliminated the forward displacement of the differential gear, allowing it to sit correctly within its housing without interference, while still retaining sufficient length to reliably couple the motor's D-shaft to the differential. Following this modification, the wheel rotated consistently and smoothly under load, and no further scratching or jamming was observed during subsequent testing.

12.4 Iterations

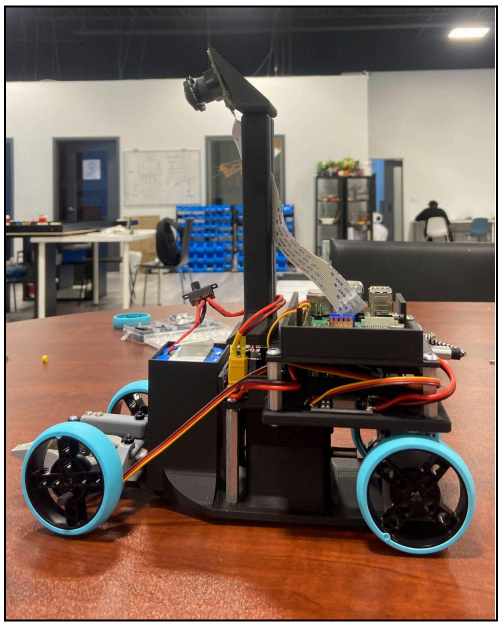
The image below shows the main robot iterations during development. Each version was tested and improved based on issues with stability, steering, and reliability.



Robot iterations, V1 through final version

12.4.1 V1 Robot — Purpose and Approach

Before the design was covered in this journal, we built a V1 prototype. Its purpose wasn't competition performance — it was to test our core assumptions (chassis layout, wheel choice, steering geometry, drivetrain) against real track conditions instead of relying on CAD alone. Building V1 let us find failure points early, while they were still cheap and fast to fix.



V1 Prototype

V1 Failures and Fixes

System	What We Tested	Failure Observed	Root Cause	Fix Applied (V2)
Chassis	Monocoque layout	Front and rear wheels were not aligned, robot angled up	Servo holder was not high enough	Increased size of chassis near the servo holder to raise front wheels
Steering	Ackermann Steering	Steering not as sharp and tires skidding	Ackermann geometry was not calculated and wheels were not placed at the pivot point	Redesigned with calculated 12° inner wheel Ackermann and wheels placed at pivot points
Differential	Motor Adapter and Differential Smoothness	Motor adapter weak, easily breaking off and small gear scratching big gear of differential	Motor adapter was a 1-2mm off size which caused the small gear to be at an angle and was not 3D printed strongly	Fixed measurements of motor adapter and printed in strong material and layer
Electronics	All components working together	Motor driver and voltage regulator overheating	Voltage regulator was not able to handle the load of the battery	Created a new custom PCB with better parts

12.5 Safety, Reliability & Known Limitations:

Hardware Safeguards

1. Power Distribution & Protection

- Battery Isolation: The battery pack is mechanically isolated in a dedicated compartment to prevent cells from shifting during impacts
- Voltage Regulation: Dual independent power rails (12.8V for motor/driver, 5V for logic) prevent servo current spikes from causing brownout elsewhere

2. Mechanical Protection

- Modular Design: The three-layer chassis allows quick replacement of any damaged section without rewiring the entire system
- Cable Management: Shielded connectors and routed wiring prevent accidental disconnects during impacts

3. Thermal Management

- Heat Sinks: Aluminum heat sinks on the DRV8871 motor driver and 5V buck converter dissipate heat under sustained load
- Component Selection: We chose high-efficiency buck converter (85–95%) over linear regulator (39%) specifically to eliminate thermal shutdown risk
- Tested Runtime: Validated that the robot runs cooler under competition conditions than during extended testing

4. Monitoring & Diagnostics

- LED Status Indicator: Visual feedback shows when the robot is ready, running, or in error state
- Serial Debugging: Real-time debug output over USB allows live monitoring of voltages, currents, and sensor readings during testing

Software Safeguards

1. Watchdog & Timeout Protection

- Heartbeat Check: If the Pi stops sending commands to the Nano for 100ms, the Nano defaults to safe stop
- Encoder Validation: Motor movement is verified via encoder feedback; if it doesn't match expected displacement, a fault is logged

2. Crash Detection & Recovery

- Wall Proximity Checks: Continuous scanning of inner/outer walls prevents the robot from driving directly into barriers
- Speed Limiting: Motor is capped at fixed speed to ensure the robot can always react to obstacles within the ~150ms vision cycle

3. Emergency Abort

- Manual Override: Pressing the start button again stops the robot mid-run
- Timeout Exit: If the robot runs for longer than expected (>120s), it automatically stops to prevent runaway behavior

12.6 Testing & Validation for Reliability

Testing & Validation for Reliability

1. Stress Testing

- Battery Discharge Cycle: Ran 15 consecutive laps to validate that the robot maintains performance throughout battery depletion
- Thermal Testing: Extended runs in warm environments confirmed no thermal shutdown occurs

- Mechanical Stress: Tested full-lock steering turns and obstacle impacts to verify no mechanical failures

2. Environmental Testing

- Lighting Conditions: Validated color detection under fluorescent, LED, and natural lighting to ensure consistent lane/obstacle identification
- Surface Friction: Tested on smooth and textured mat surfaces to confirm steering response is predictable
- Electrical Noise: Verified that WiFi interference or electromagnetic noise doesn't affect motor or servo control

3. Redundancy & Fallback

- Dual PCB Design: A second PCB is available as a drop-in replacement if the primary fails
- Encoder Fallback: If the stepper/motor feedback becomes unreliable, fixed-time movements provide a basic fallback
- Camera Recalibration: Color detection parameters can be re-tuned on-field in under 5 minutes without requiring hardware changes

13.0 Reproducibility & GitHub

This documents the design, build, and code for our WRO robot — including the mechanical models, custom PCB/electronics, full parts list, and the engineering journal tracking our design decisions along the way. Use the sections below to navigate to whatever you need, whether that's rebuilding the robot from scratch or understanding why we made the choices we did.

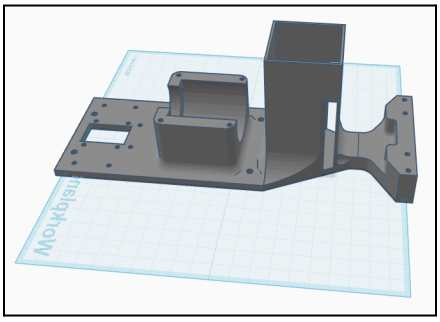
13.1 How to Assemble the Robot

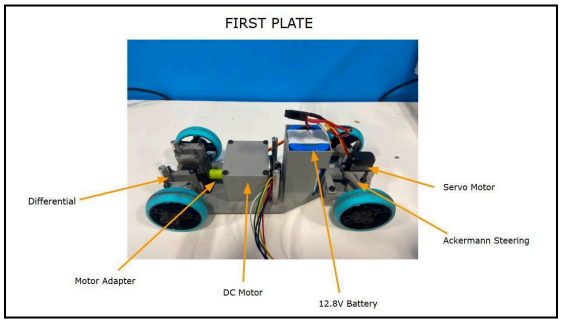
Chassis

Three stacked, modular plates — any layer pulls off without touching the others.

Base Plate

- Steering holder: trapezoid front section, 4 holes, servo cutout.
- Battery holder: front-mounted, low, counterbalanced rear motor.
- Motor mount: mid-rear, upper/lower halves clamp the motor.
- Rear axle points: cutouts + mounts for axle supports.

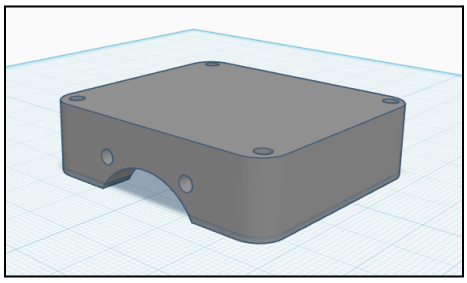


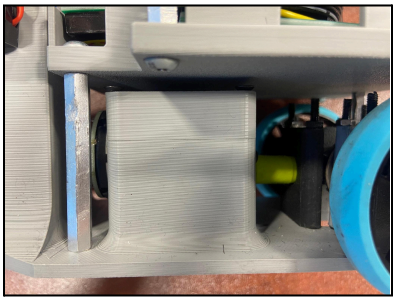


Base Plate — CAD / Base Plate — Installed

Motor Mount — Top Half

Clamps over the motor, bolts into the lower half.

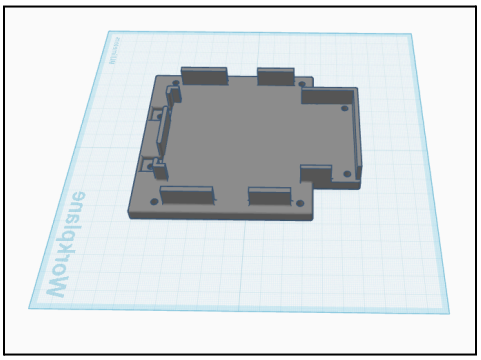


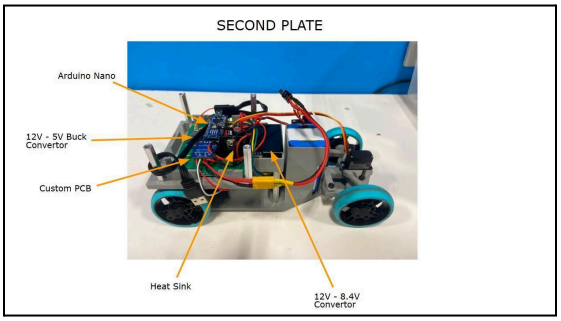


Motor Mount Top — CAD / Motor Mount Top — Installed

Second Story — PCB Holder

Holds our custom PCB. The main rectangle seats the PCB, the second smaller one houses the servo regulator (also fits our old PCB as backup).

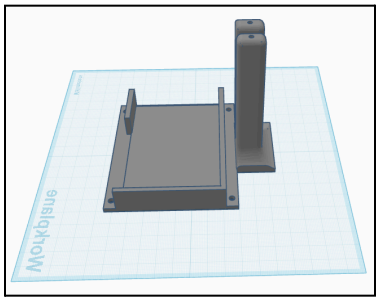


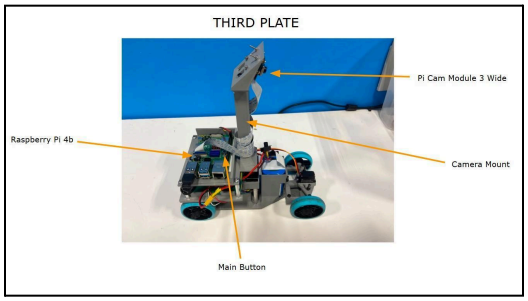


Second Story — CAD / Second Story — Installed

Third Story — Pi Holder

Holds the Pi. Partial wraparound wall, open sides for cables, includes the Camera Stand which raises the camera for a clear view of the mat.

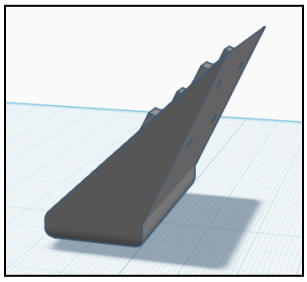


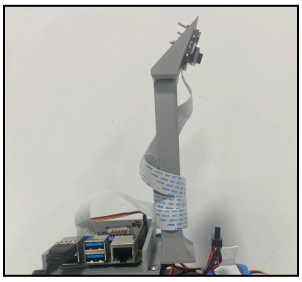


Third Story — CAD / Third Story — Installed

Camera Mount

Sits atop the stand and holds the camera. Standalone piece so angle adjusts without touching the stand or top plate. Set to 155°.

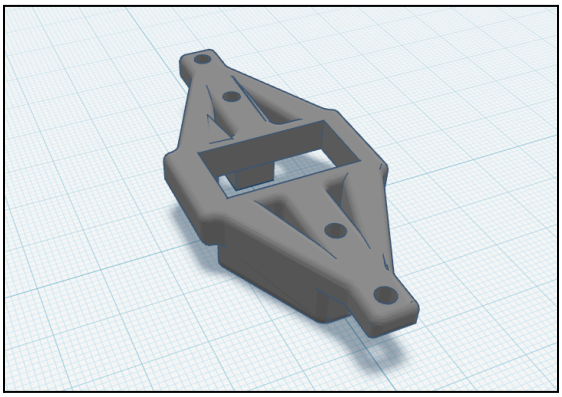


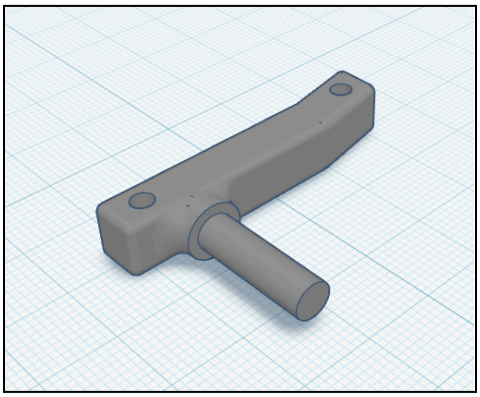


Camera Mount — CAD / Full Camera Assembly — Installed

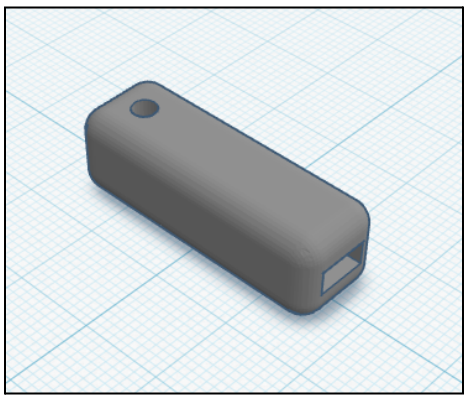
Steering Assembly

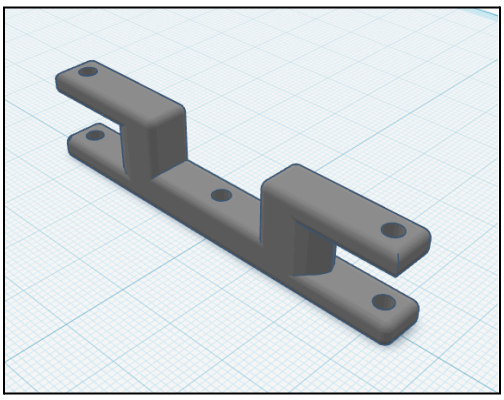
- Steering holder: everything mounts here, servo inverted to line up with knuckles.
- Steering Arm: fixed at ~12° Ackermann angle, nylon spacers cut friction.
- Steering T: extends servo output to the steering bar.
- Steering Cross Link: links both knuckles, converts servo rotation into steering.



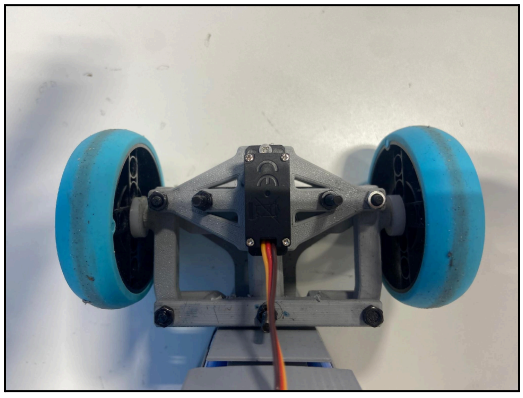


Servo Holder / Steering Arm





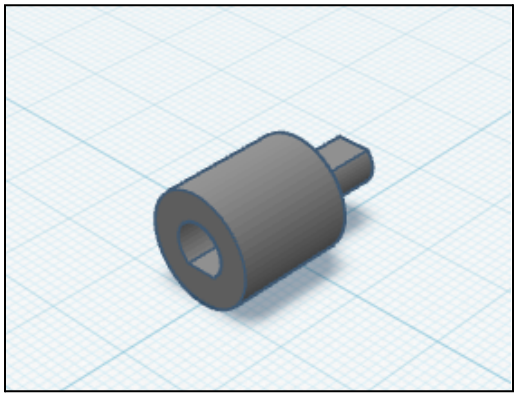
Steering T / Steering Cross Link

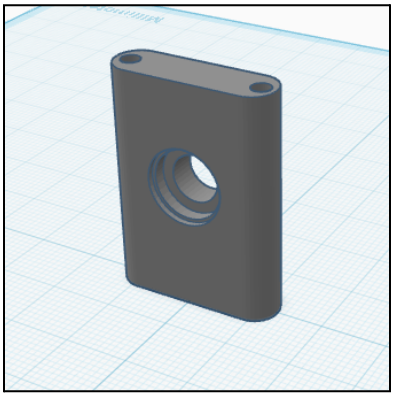


Full Steering Assembly — Installed

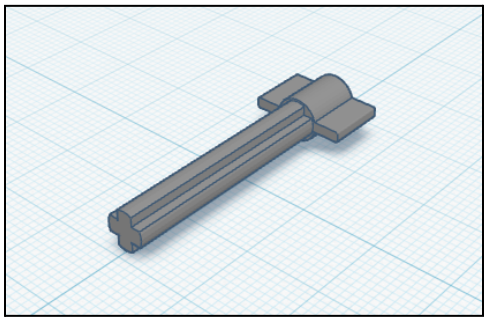
Rear Axle Assembly

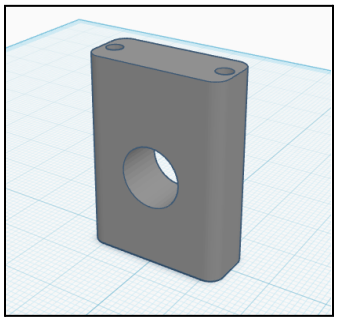
- Motor adapter: matches motor's D-shaft to the differential's square input.
- Gear holder: connects to the differential to hold it in place.
- Axle supports: hold the axles in place via bearings, keep them aligned and stop sagging.
- Axles: custom metal axles adapt diff output to LEGO wheels.
- VEX spacers: added on the rear axles to push the wheels further outward, giving clearance so they don't rub against the chassis/body.



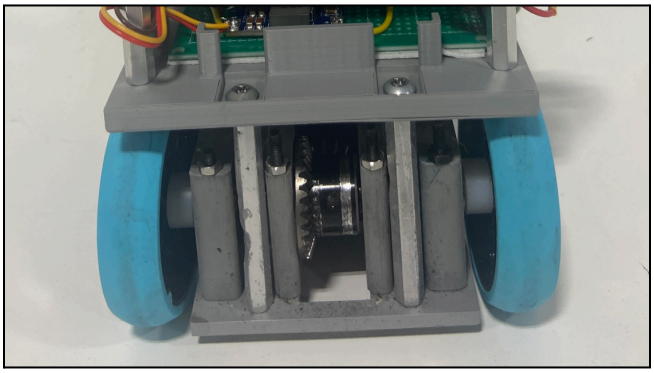


Motor Adapter / Gear Holder





Axles / Axle Support



Full Rear Axle Assembly — Installed

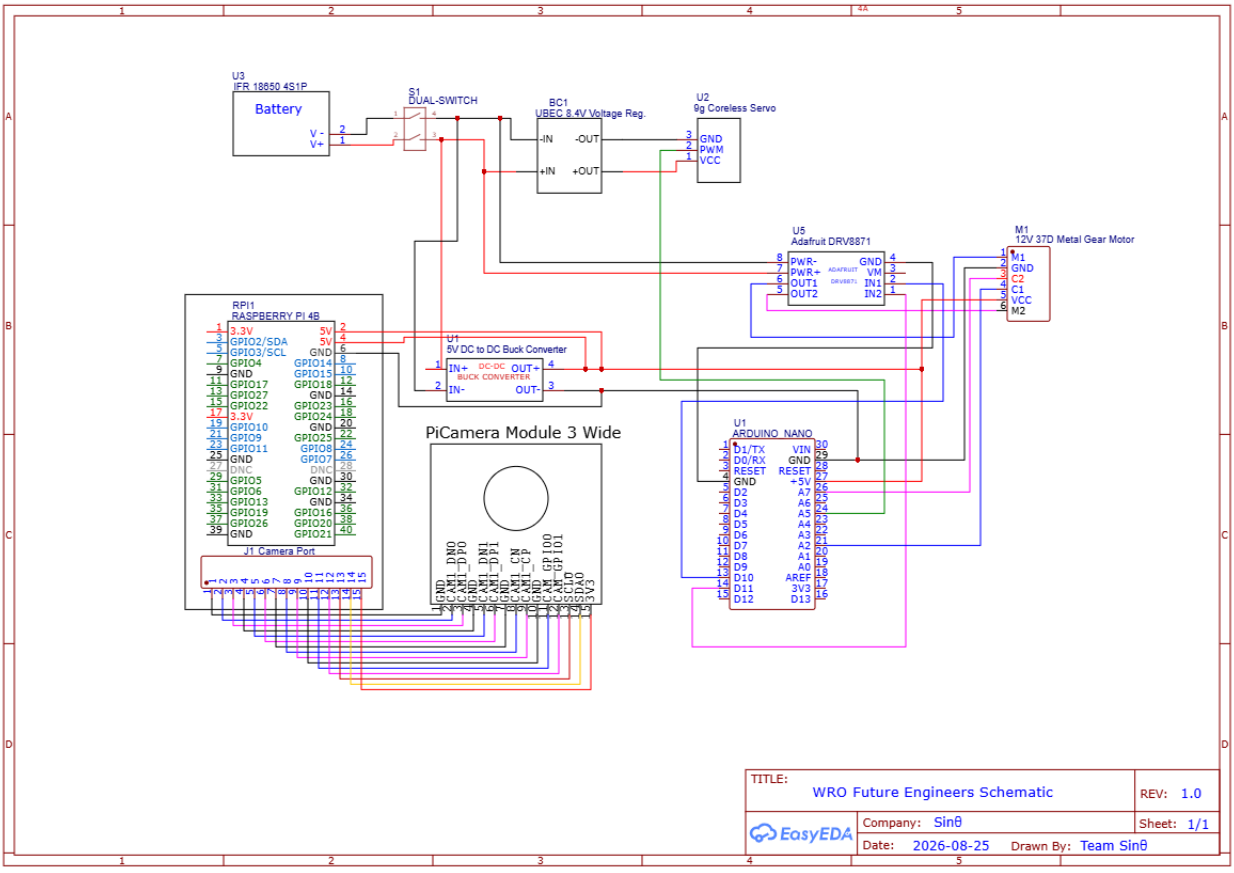
- Attach LEGO SPIKE Prime wheels to rear axles and front steering system
- Validation: Wheels spin freely with no friction; differential engages smoothly

13.2 How to Build the Custom PCB

Step 1: Gather All Components

- See complete Parts List
- Total cost: $443.62
- Verify all items received before starting

Follow along with this schematic of the entire electrical architecture for the next steps



PCB Schematic — WRO Future Engineers, Team Sinϴ

Step 2: Solder Battery Connections

- Solder battery pack leads to a switch
- Solder switch positive and negative ends to the PCB board
- Time estimate: 20 minutes

Step 3: Install Power Regulators

- Mount 5V buck converter to PCB
- Solder battery(Positive) to 5V Input positive on buck converter
- Solder battery(Negative) to 5V Input negative on buck converter
- Solder Buck converter OUTPUT (Positive) to 5V Rail
- Solder Buck converter OUTPUT (Negative) to Ground Rail
- Mount 8.4V UBEC voltage regulator
- Solder Battery Positive to Input positive on UBEC
- Solder Battery Negative to Input negative on UBEC
- Connect UBEC Output Positive to Servo VCC
- Connect UBEC Output Negative to Servo GND
- Time estimate: 60 minutes

Step 4: Wire Motor & Servo

- DRV8871 Motor Driver: Connect Power Positive and Negative to Battery Positive and Negative
- DRV8871 Motor Driver: Connect Output 1 and 2 to Motor M1 and M2
- DRV8871 Motor Driver: Connect In1 and In2 to Nano
- DRV8871 Motor Driver: Connect GND to Nano GND
- Servo: Connect Servo PWM to Analog Pin on Nano
- Motor encoder: Connect 5V, GND, and pulse line to Nano
- Time estimate: 80 minutes

Step 5: Wire Raspberry Pi & Arduino Nano

- Power Pi from 5V buck converter (5V and GND)
- Power Nano from 5V buck converter (5V and GND)
- Serial connection: Pi UART TX → Nano RX; Pi UART RX → Nano TX
- Camera: Connect 15 Camera 3 to Pi's CSI/DSI connector
- Time estimate: 15 minutes

Step 6: Mount Electronics in Chassis

- Place battery in center of base plate (battery holder)
- Mount PCB on second story PCB holder
- Mount Raspberry Pi on third story Pi holder
- Secure camera on camera stand at 155° angle
- Time estimate: 10 minutes
- Mechanical check: No loose wires; all components secured

Electrical Testing Checkpoint:

Battery reads 12.8V
5V rail reads 5.0V ± 0.1V

8.4V servo rail reads 8.4V ± 0.2V
All GND connections are solid (multimeter reads 0Ω between points)
Motor driver responds to PWM commands (test with function generator)
Servo moves smoothly when sent PWM signal (use Nano test code)
Camera captures frames

13.3 Software Setup

Step 1: Set Up Raspberry Pi

1) What You Need

- Raspberry Pi (4 or 5 recommended) with Raspberry Pi OS
- Raspberry Pi Camera Module, connected to the CSI port
- Arduino Nano connected via USB
- Push button + status LED wired to GPIO (pins 26 and 16 by default)

2) Update the System

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

3) Enable the Camera

```
sudo raspi-config
```

- Go to Interface Options → Camera → Enable
- Reboot when prompted

4) Find the Arduino's Serial Port

The Nano usually shows up as /dev/ttyUSB0 or /dev/ttyACM0. Check which one:

```
ls /dev/tty*
```

Plug/unplug the Nano and re-run the command — whichever entry appears/disappears is the right port. Update SERIAL_PORT_NAME in the code if it's different from /dev/ttyUSB0.

Give your user permission to use serial ports, then reboot:

```
sudo usermod -a -G dialout $USER
sudo reboot
```

5) Install System Dependencies

```
sudo apt install -y python3-pip python3-venv python3-opencv python3-picamera2
```

opencv and picamera2 are installed as system packages here since they rely on hardware libraries that pip struggles to install cleanly on a Pi.

6) Create a Virtual Environment

Create it with access to the system packages just installed:

```
python3 -m venv --system-site-packages venv
source venv/bin/activate
```

Run source venv/bin/activate again anytime you open a new terminal.

7) Install Python Dependencies

```
pip install pyserial numpy gpiozero RPi.GPIO
```

Package	What it's for
opencv (system)	Vision pipeline — colour masks, contours, steering math
picamera2 (system)	Reading frames from the Pi camera
numpy	Array math used throughout the vision pipeline
pyserial	Talking to the Arduino Nano over USB
RPi.GPIO	Reading the start button
gpiozero	Controlling the status LED

8) Wiring Check

Component	GPIO Pin (BCM)
Start button	26
Status LED	16

9) Running the Code

```
source venv/bin/activate
python3 ObstacleChallenge.py
```

The status LED turns on, and the program waits for the start button to be pressed.

Step 2: Load Arduino Firmware and Verify Pi-Nano Communications

- Test serial communication between Raspberry Pi and Arduino Nano
- Download the Arduino motor controller code
- Time estimate: 15 minutes

Step 3: Tune Color Detection

- Run LAB Adjuster
- For each color (blue, green, red, orange, pink):
    - Show actual obstacle/line on track
    - Adjust LAB thresholds until detection is 100%+ accurate, in various positions and lighting conditions
    - Save thresholds to Obstacle Challenge
- Time estimate: 20-30 minutes
- Test: Run on practice mat; verify all colors detected reliably

Software Testing Checkpoint

Raspberry Pi boots and runs Python scripts
Arduino Nano receives and executes motor commands
Serial communication between Pi and Nano is reliable (no dropped packets)
Camera captures clear frames with correct white balance
Color detection identifies all 5 colors (blue, green, red, orange, pink) reliably
No Python syntax errors or import failures

13.4 First Test Run on Practice Mat (30 minutes)

Step 1: Safety Check

- Robot is on a raised stand (not on ground yet)
- Motor is powered but wheels can spin freely
- No loose wires or parts
- Camera has clear view
- All emergency stops are accessible

Step 2: Test Wall Following

- Place robot on mat with left wall nearby
- Run wall-following mode only (no obstacles)
- Expected: Robot hugs the left wall, maintains consistent distance
- Measure: Distance to wall should vary <2cm throughout lap
- If fails: Adjust AREA_WALL_KP and AREA_WALL_KD in code (Open Challenge, Obstacle Challenge)

Step 3: Test Obstacle Avoidance

- Place green obstacle in path
- Run obstacle avoidance mode
- Expected: Robot steers left to avoid, then returns to wall
- Measure: Minimum clearance from obstacle should be >5cm
- Test with: Green, Red, Orange obstacles at different distances (20cm, 50cm, 80cm)
- If fails: Adjust GREEN_PILLAR_KP, RED_PILLAR_KP, PILLAR_STEER_OFFSET_DEG

Step 4: Test Complete Run

- Run full state machine (wait → warmup → drive laps → park)
- Expected: Robot completes 3 laps + parks in pink zone
- Measure: Lap time, parking accuracy, any crashes
- Record results in testing log

Step 5: Run Combinations

- Run the Random Combination Generator to test all combinations of obstacles, to tweak the code for overall consistency

First Test Checkpoint:

Robot can wall-follow for 1 full lap without crashing
Robot avoids obstacles without overshooting
Motor encoder ticks match expected wheel rotations
Processing latency is consistent ~150ms
No thermal issues (touch PCB, motor driver, regulators — all cool)

13.4 Future Improvements

1. Begin to push code directly from the Raspberry Pi through Git in order to build a commit history.
2. Implement Dynamic Ranges to improve obstacle, line, and parking block detection in a multitude of lighting conditions.
3. Improving on our Acceleration and Deceleration code to a point that it becomes as reliable as fixed code, therefore we can improve our lap times overall.

Conclusion

When we set out to build Team SinΘ's robot for WRO Future Engineers 2026, our goal was never simply to build the fastest machine on the track. It was to design, build, and validate a complete autonomous system from scratch, using the lessons of our previous teams (VHR Motors and The Robertos) as a foundation rather than a starting point we would repeat unchanged.

Across mechanical design, we moved from mismatched wheel setups and 3D printed drivetrain components to a uniform SPIKE Prime wheel configuration, a metal differential on direct drive, and calculated Ackermann steering. Each decision traced back to a specific failure mode one of our prior robots had already exposed. In power and sensor architecture, two full PCB failure cycles, including a 7805 thermal shutdown dissipating 15.6W and a subsequent motor driver and servo power isolation failure, forced us to rebuild our electrical system around a buck converter and an isolated servo UBEC. Those failures became the foundation of the stable architecture that ultimately ran our final robot. In software, committing to a single camera, LAB colour space vision pipeline let us hold our sense think act loop to roughly 150ms per frame, fast enough to support a priority based arbitration system that could resolve outer wall safety, inner wall damping, obstacle avoidance, and wall following into one coherent decision every cycle.

The result was a robot that met or exceeded every benchmark we set for ourselves: lap times of 28s (Open) and 70s (Obstacle) against our 45s and 90s targets, parking accuracy of ±1cm against a ±3cm target, and a 90% success rate across 15 consecutive validation laps, all built for $443.62, under our $500 budget.

More important than any individual metric, though, is what the process taught us. Nearly every major design decision in this journal exists because something failed first: a scrubbing tire, an overheating regulator, a burned out servo, a misaligned gear. Treating those failures as data rather than setbacks, and being willing to redesign rather than patch over a symptom, is what lets us turn a season of iteration into a robot we trust to run consistently, not just quickly.

Looking ahead, we see clear room to keep improving: closed loop speed control tied more tightly to the encoder feedback we already collect, further refinement of our obstacle avoidance gains across a wider range of pillar placements, and continued hardening of our PCB against edge cases we have not yet triggered. But we are proud that every subsystem in this robot, mechanical, electrical, and software, was built, broken, diagnosed, and rebuilt by our own hands, and that this journal reflects not just what we built, but why.

We would like to thank our coaches, mentors, and the WRO community, including the teams whose documentation and design choices inspired parts of our own approach, for helping make this season possible.

References & Technical Sources

Ref.	Title to put in Journal	Website / Organization
[1] WRO 2026 Future Engineers – General & Game Rules	WRO Association – 2026 Season	Competition dimensions, weight, game requirements, robot constraints
[2] WRO 2026 Future Engineers – Documentation Rubric	WRO Association – 2026 Season	Engineering-documentation expectations and rubric
[3] Raspberry Pi 4 Model B – Technical Specifications	Raspberry Pi 4 Technical Specifications	Raspberry Pi specifications, power requirements, interfaces, CSI connection
[4] Raspberry Pi Camera Module 3 – Documentation	Raspberry Pi Camera Documentation	Pi Camera 3 operation, resolution, autofocus, camera specifications
[5] Raspberry Pi Camera Module 3 – Product Brief	Official Camera Module 3 Product Brief	Camera sensor, dimensions, CSI-2, IMX708 and hardware specifications
[6] Raspberry Pi Camera Module 3 Wide – Technical Specifications	Raspberry Pi Camera Module 3	Wide camera FOV and specifications; particularly relevant because Sinθ uses the Wide version
[7] Arduino Nano – Hardware Documentation	Arduino Nano Documentation	Nano dimensions, pinout, schematic and hardware specifications
[8] DRV8871 – 3.6-A Brushed DC Motor Driver with Internal Current Sense (PWM Control)	Texas Instruments DRV8871 Documentation and Datasheet	Motor-driver voltage/current limits, PWM operation, protection and thermal specifications
[9] OpenCV – Color Space Conversions	OpenCV Documentation	LAB conversion and cvtColor() used in your computer-vision pipeline

[10] OpenCV – Image Processing Documentation	OpenCV Image Processing Documentation	Thresholding, masks, contours, image processing and vision algorithms
[12] Ackermann Steering Geometry	X-Engineer – Ackermann Steering Geometry	Inner/outer steering angles, wheelbase, track width and turning-radius theory